Software Engineering
5430
Spring 2023

by:
Abdelnasser Ouda
Computer Science Department
Email: abdelnasser . ouda @ unt.edu
Discovery Park F203

Unit 2: Requirements Elicitation - RE

- Introduction and Overview
- Requirements Elicitation Concepts
- Requirements Elicitation Methods
- Requirements Elicitation Activities
- Software Requirements Specification (SRS)
- Requirements Traceability and Validation

Requirements Elicitation

Introduction and Overview

What is UML?

- Unified Modeling Language (UML) is the standard modeling language for software and systems development, by OMG. Current version, UML 2.5.
- OMG, the Object Management Group is an international, open membership, not-for-profit technology standards consortium, founded in 1989.
- Commercial tools: **MagicDraw (No Magic)**, Rational (IBM), **MS Visio**
- Open Source/Free tools: ArgoUML, StarUML, Dia, online Gliffy, Umbrello, MS Visio (for UNT students)

Views of System Model

We use Models to describe Software Systems

- Object model: What is the structure of the system?
  What are the objects and how are they related?
  (Class Diagram)
- Functional model: What are the functions of the system?
  How is data flowing through the system?
  (Use Case Diagram)
- Dynamic model: How does the system react to external events?
  How is the event flow in the system ?
  (Sequence, Statechart, Activity Diagrams)
- System Model: Object + functional + dynamic (models)

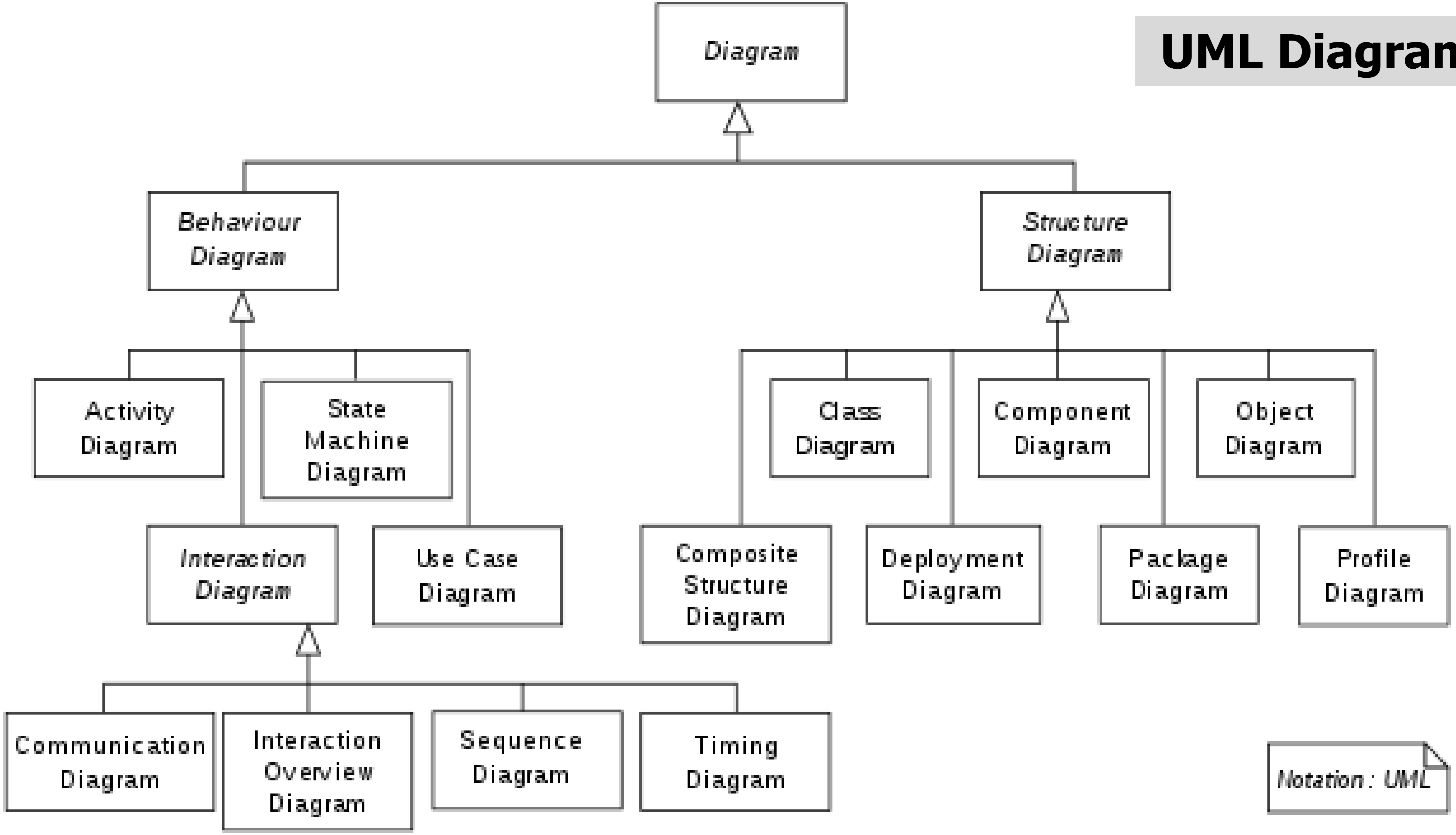
Other UML Notations

UML provides many other notations, for example

- Deployment diagrams for modeling configurations
- Component diagrams for modeling system components interfaces
- Package diagrams for organization groups of classes & components
- Timing diagrams showing the behaviors of one or more objects throughout a given period of time.

- There is also a language to constrain the systems that can be instantiated from models. The language that has been designed to constrain UML models is called OCL (Object constraint language). (Unit5)

UML Diagrams

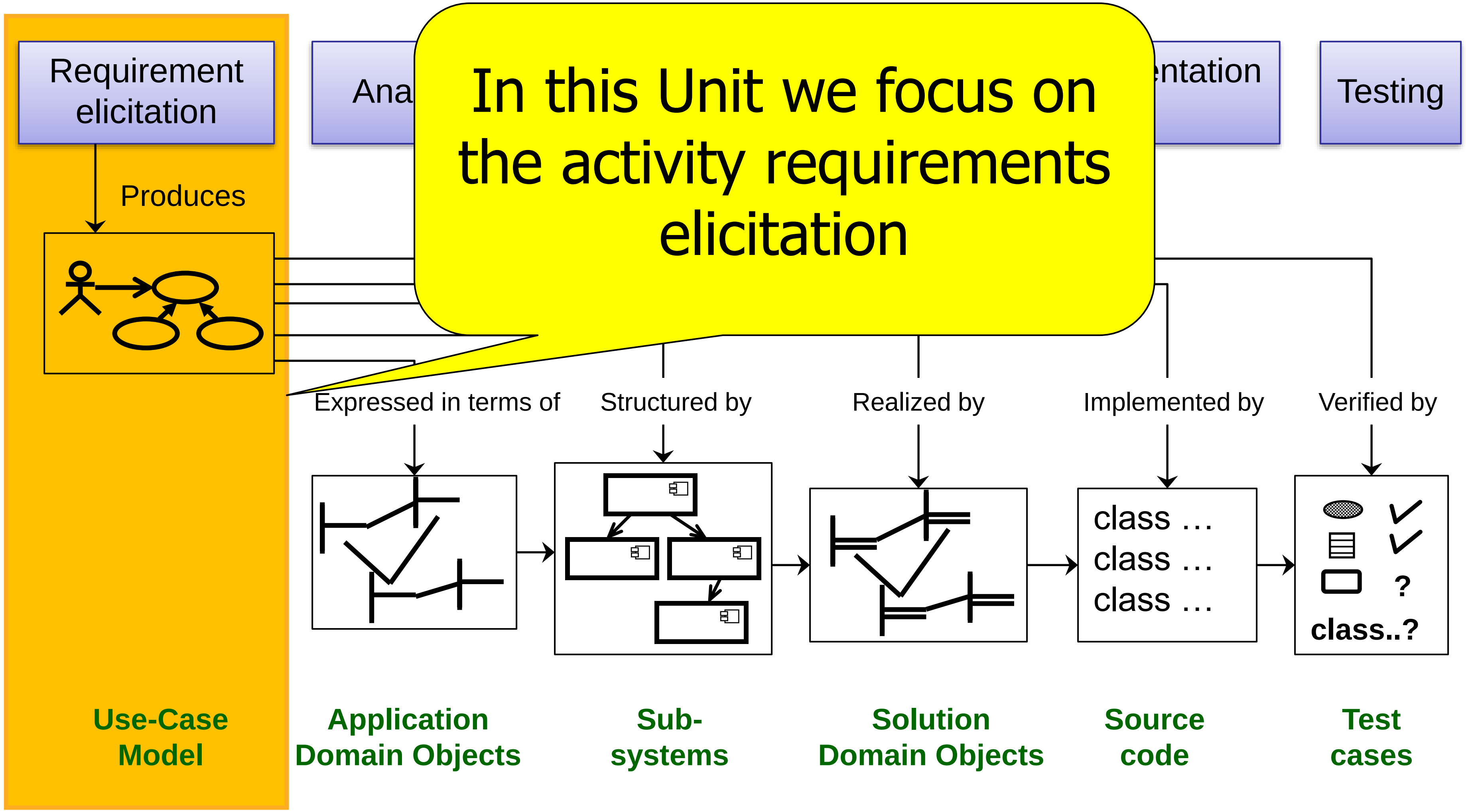


UML diagrams can be categorized hierarchically as shown in the above class diagram.

What is the Requirements Elicitation?

- The least technical activity of software development lifecycle
- Demands social, communication and managerial skills
- Delivers a (mostly narrative) definition of user requirements

Requirements Elicitation and SDLC



What does the Customer say?

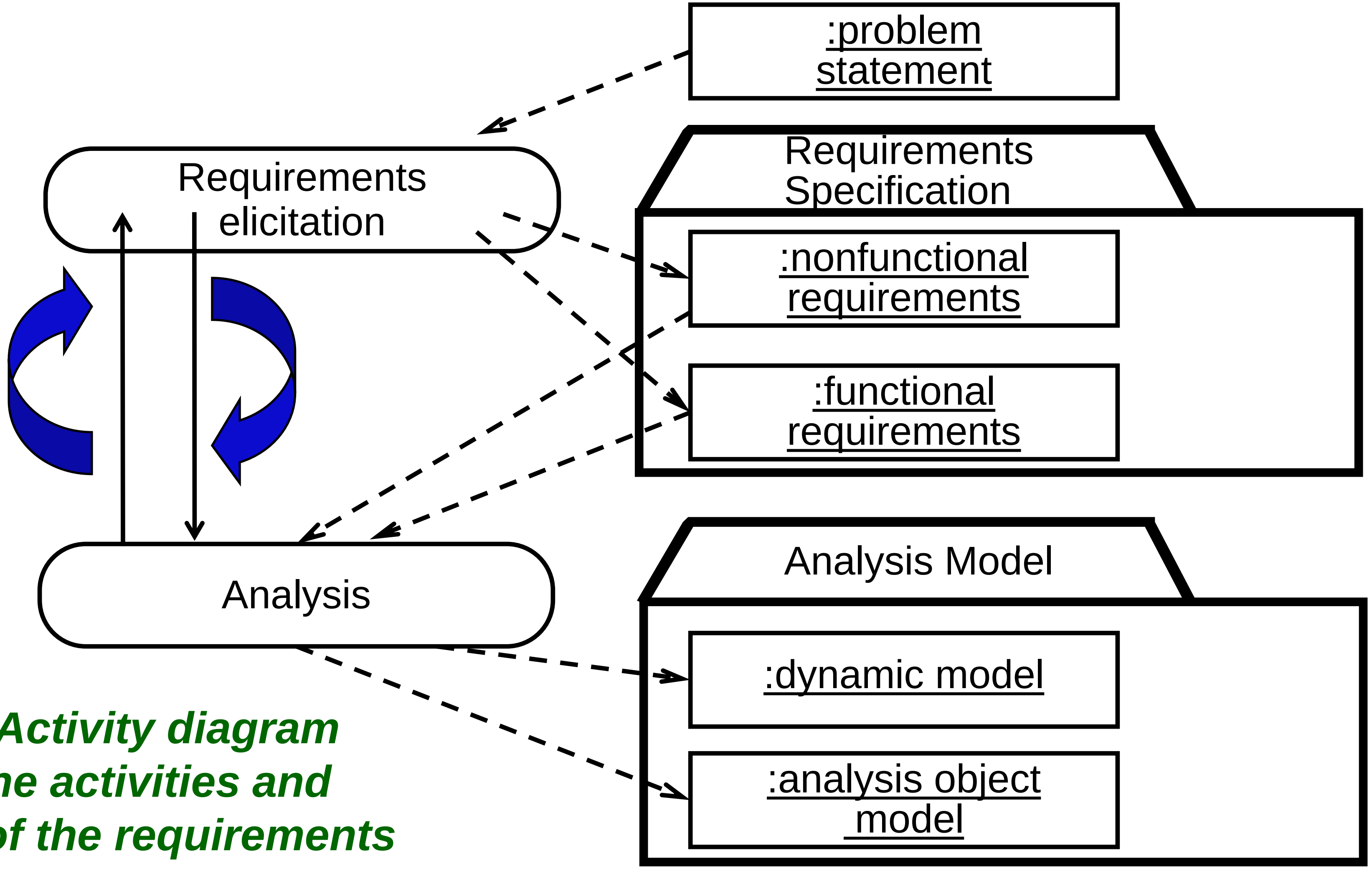
First step in identifying the Requirements: System identification

- Two questions need to be answered:
  1. How can we identify the purpose of a system?
  2. What is inside, what is outside the system?
- These two questions are answered during requirements elicitation and analysis (**Requirements Identification**).

Requirements Specification vs Analysis Model

- Both focus on the requirements from user's view of the system
- Requirements elicitation:
  - Definition of the system in terms understood by the customer ("Requirements specification" or "Software Requirements Specification" SRS)
  - The requirements specification uses natural language (derived from the problem statement)
- Analysis:
  - Definition of the system in terms understood by developer(s) (Technical specification, "Analysis model")
  - Analysis model uses a formal or semi-formal notation, UML.

Requirements Process



This is an Activity diagram showing the activities and products of the requirements engineering process

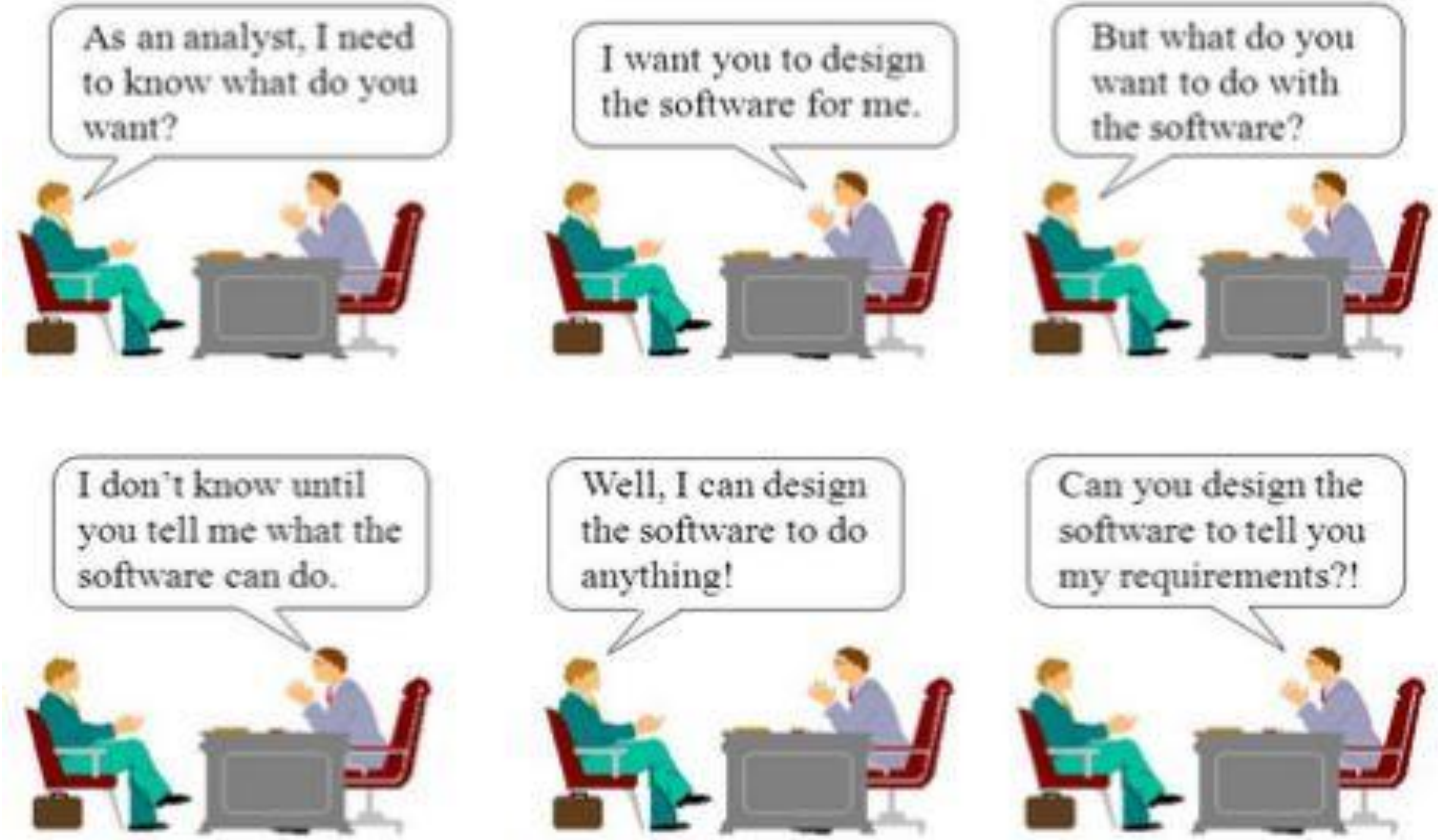
Understand the Problem Statement

- The problem statement is developed by the client
  - Also called scope statement
- A problem statement describes
  - The current situation
  - The functionality the new system should support
  - The environment in which the system will be deployed
  - Deliverables expected by the client
  - Delivery dates
  - Criteria for acceptance test.

Ingredients of a Problem Statement

- Current situation
  - The problem to be solved
  - Description of one or more scenarios
- Requirements
  - Functional and nonfunctional requirements
  - Constraints
- Target environment
  - The environment in which the delivered system should perform a specified set of system tests
- Project schedule
  - Major milestones that involve interaction with the client including deadline for delivery of the system
- Client acceptance criteria
  - Criteria for the system tests.

Challenges of Requirements Elicitation



Types of Requirements

- **Functional requirements**
  - Describe the interactions between the system and its environment independent from the implementation
  - The environment includes the user and any other external system with which the system interacts.
  - For example, consider the following functional requirements for SatWatch.

SatWatch Functional requirements example

- SatWatch is a wrist watch that displays the time based on its current location.
- SatWatch uses GPS satellites (Global Positioning System) to determine its location and then convert this location into a time zone.
- The watch owner never needs to reset the time. SatWatch adjusts the time and date displayed as the watch owner crosses time zones and political boundaries. For this reason, SatWatch has no buttons or controls available to the user.
- SatWatch has a two-line display showing, on the top line, the time (hour, minute, second, time zone) and on the bottom line, the date (day, date, month, year).
- The display technology used is such that the watch owner can see the time and date even under poor light conditions.
- When political boundaries change, the watch owner may upgrade the software of the watch using the WebifyWatch device (provided with the watch) and a personal computer connected to the Internet.

The above functional requirements focus only on the possible interactions between SatWatch and its external world
The above description does not focus on any of the implementation details, e. g., processor, language, display technology

Types of Requirements - Nonfunctional requirements

- **Nonfunctional requirements**
  - Describe aspects of the system that are not directly related to the functional behavior of the system.
  - Nonfunctional requirements can be
  - Quality requirements, e.g.,
    - Usability
    - Reliability
    - Performance
    - Supportability
  - Or Constraints such as,
    - Implementation requirements
    - Interface requirements
    - Operations requirements
    - Packaging requirements
    - Legal requirements

SatWatch non-Functional requirements example (next slide)

Quality requirements for SatWatch

- Any user who knows how to read a digital watch and understands international time zone abbreviations should be able to use SatWatch without the user manual. [Usability requirement]
- As the SatWatch has no buttons, no software faults requiring the resetting of the watch should occur. [Reliability requirement]
- SatWatch should display the correct time zone within 5 minutes of the end of a GPS blackout period. [Performance requirement]
- SatWatch should measure time within 1/100th second over 5 years. [Perf.]
- SatWatch should display time correctly in all 24 time zones. [Perf.]
- SatWatch should accept upgrades to its onboard via the Webify Watch serial interface. [Supportability requirement]

Constraints for SatWatch

- All related software associated with SatWatch, including the onboard software, will be written using Java, to comply with company policy. [Impl.]
- SatWatch complies with the physical, electrical, and software interfaces defined by WebifyWatch API 2.0. [Interface requirement]

What should not be in the Requirements?

- System structure, implementation technology
- Development methodology
- Development environment
- Implementation language
- Reusability

It is desirable that none of these above are constrained by the client. Fight for it!

Requirements Elicitation Tasks

The FRIEND system

In FRIEND field officers, such as, a police officer or a fireman, have access to a wireless computer that enables them to interact with a dispatcher. The dispatcher in turn can visualize the current status of all its resources, such as, police cars or trucks, on a computer screen and dispatch a resource by issuing commands from a workstation.

Requirements Elicitation Tasks

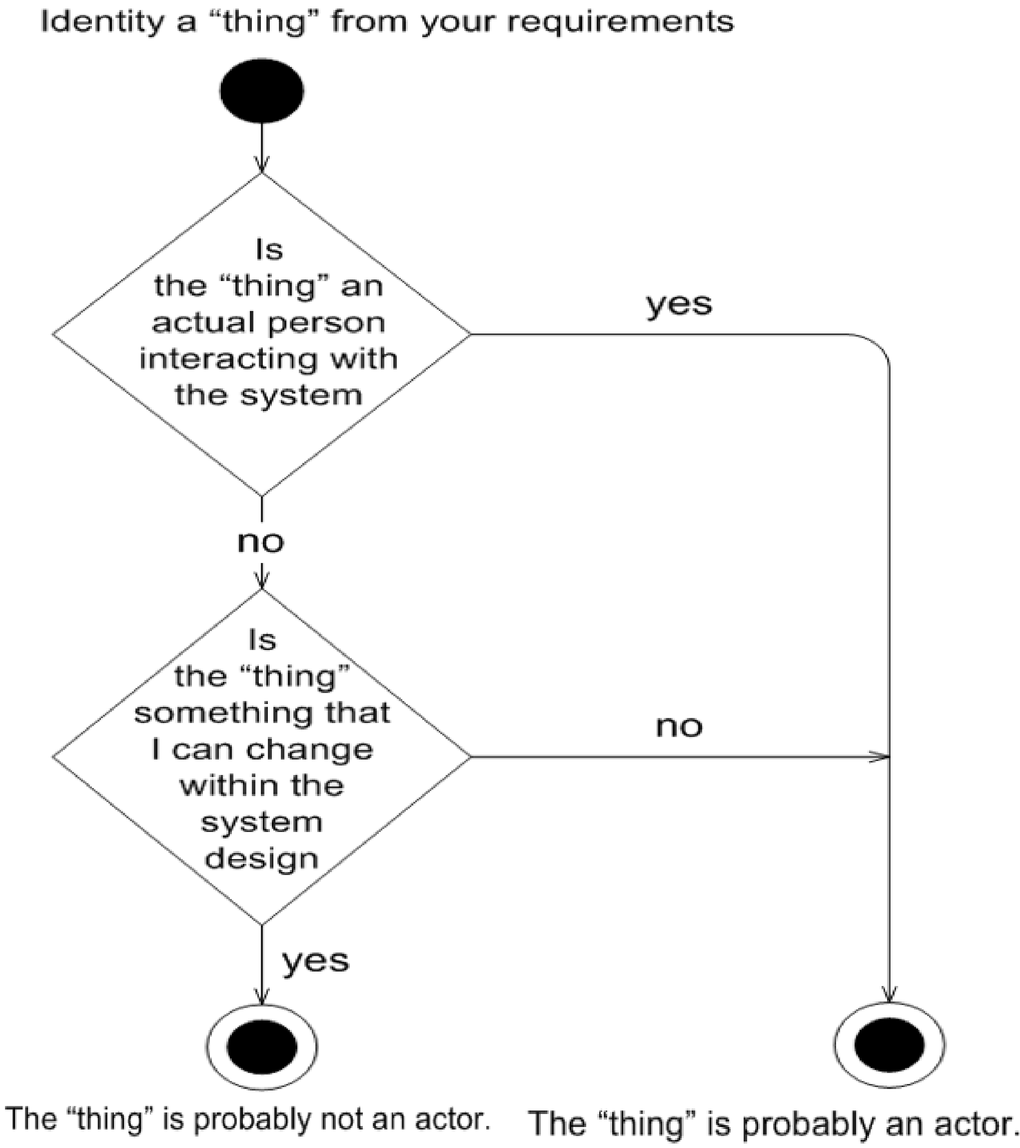
- During requirements elicitation, software engineers access many different sources of information.
- These information will be represented in structured forms by applying the following tasks:
  - Identifying actors: software engineers identify the different types of users the future system will support.
  - Identifying scenarios: software engineers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
  - Identifying use cases: software engineers derive from the scenarios a set of use cases that completely represent the future system.
  - Refining use cases: software engineers ensure the completeness of requirements specification by detailing each use case and describing the behaviour of the system in the presence of errors and exceptional conditions.
  - Identifying relationships among use cases: software engineers ensure the consistence of the requirements specification by factoring out the common functionality in the use case model.
  - Identifying nonfunctional requirements: developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.

Identifying Actors

- Actors represent external entities that interact with the system.
- An actor can be human or an external system.
- **An actor** is drawn in UML notation using either a "stick man" or a stereotyped box, and is labeled with an appropriate name.
- The FieldOfficer is an actor to the Friend system
- The FieldOfficer interacts with the system and is not part of the system; therefore, the FieldOfficer is defined as an actor.

Identifying Actors …

- Consider this simple technique for indentifying a "thing" that you've found in your requirements and how to decide whether it is an actor or not.
  - Anything that you can affect during system design and have some control over, is likely to be considered part of the system.
  - Be careful when it comes to people, some people get benefit from the system and they are not actors.
- Note that, Timer is often an actor.



Identifying Actors …

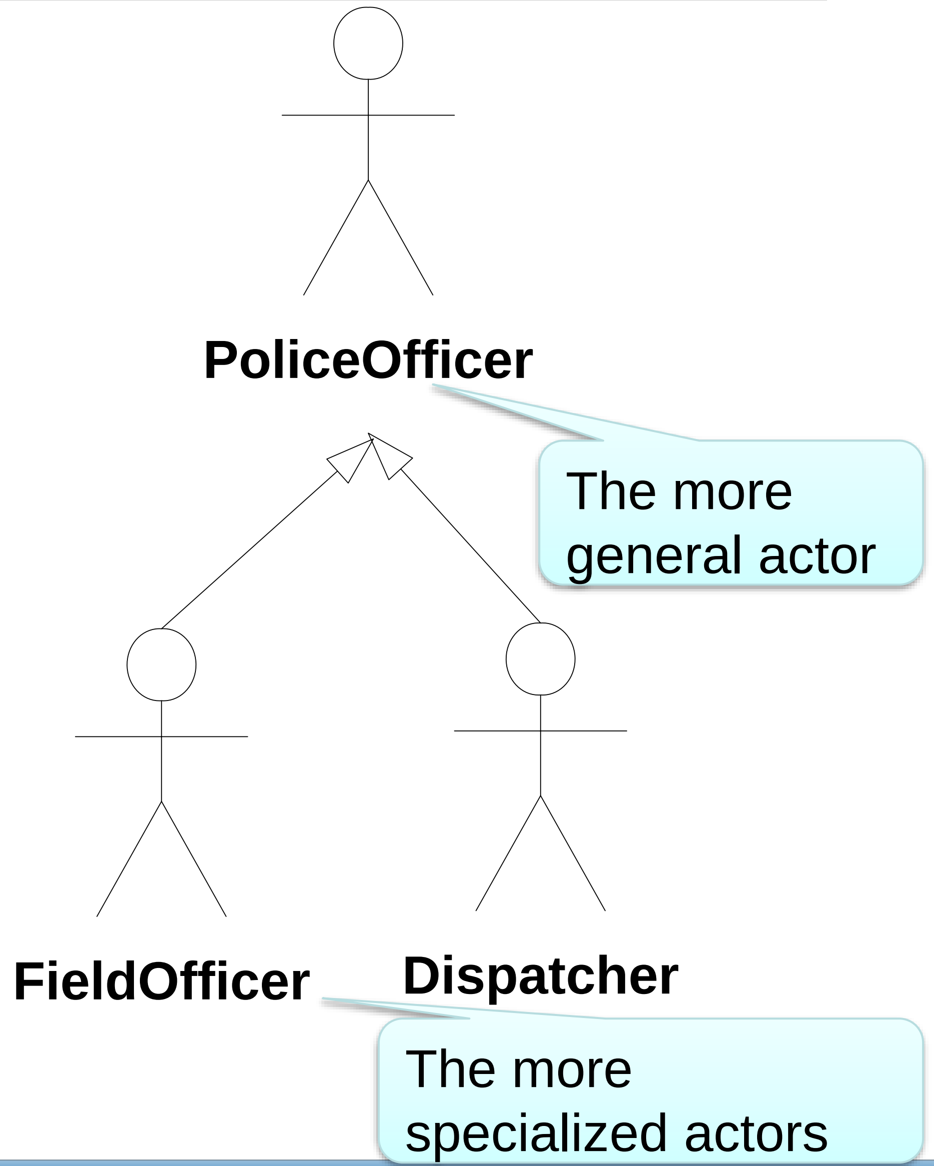
- Asking the following questions will help you indentify actors.
  - Who or what uses the system?
  - What roles do they play in the interaction?
  - Who installs the system?
  - Who or what starts and shuts down the system?
  - Who maintains the system?
  - What other systems interact with this system?
  - Who or what gets and provides information to the system?
  - Does anything happen at a fixed time?

Identifying Actors …

- In terms of indentifying actors, remember following points.
  - Actors are always **external** to the system – they are therefore outside your control.
  - Actors **interact directly** with the system.
  - Actors represent **roles** that people and things play in relation to the system, not specific people or specific things.
  - Each actor has a **unique name** and description.
- Accordingly:
  - The **Dispatcher** is also an actor to the FRIEND system

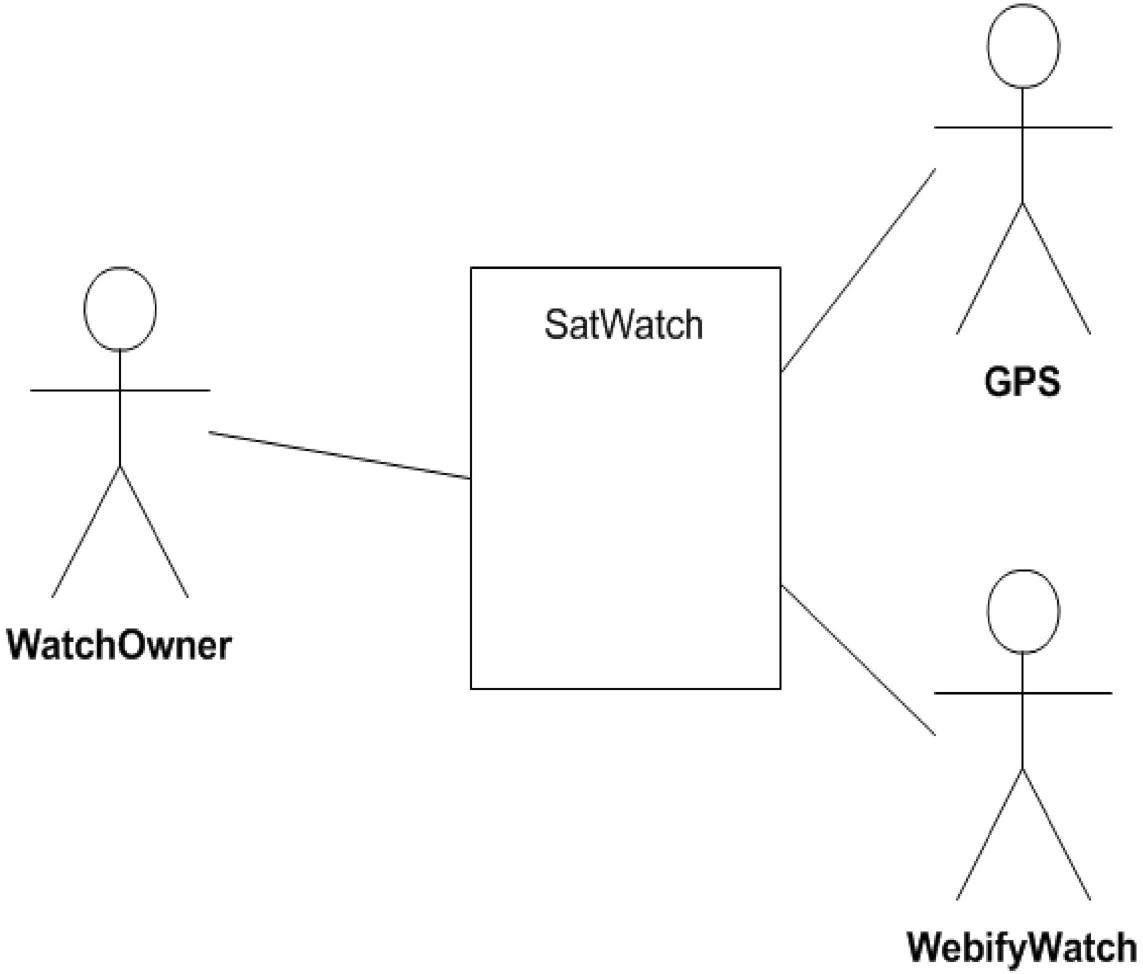
Generalization between Actors

- During actors' analysis process we may find number of actors have common characteristics.
- It is not uncommon to see multiple actors have common roles or purposes interacting with the system.
- The **FieldOfficer** actor and **Dispatcher** actors are special kind of **PoliceOfficer** actor.



Actors for the SatWatch system

- WatchOwner moves the watch (possibly across time zones) and consults it to know what time it is.
- SatWatch interacts with GPS to compute its position.
- WebifyWatch upgrades the data contained in the watch to reflect changes in time policy (e. g., changes in daylight savings time start and end dates).



Requirements Elicitation Tasks

- During requirements elicitation, software engineers access many different sources of information.
- These information will be represented in structured forms by applying the following tasks:
  - Identifying actors: software engineers identify the different types of users the future system will support.
  - Identifying scenarios: software engineers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
  - Identifying use cases: software engineers derive from the scenarios a set of use cases that completely represent the future system.
  - Refining use cases: software engineers ensure the completeness of requirements specification by detailing each use case and describing the behaviour of the system in the presence of errors and exceptional conditions.
  - Identifying relationships among use cases: software engineers ensure the consistence of the requirements specification by factoring out the common functionality in the use case model.
  - Identifying nonfunctional requirements: developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.

Identifying Scenarios

- A **scenario** is "a narrative description of what people do and experience as they try to make use of computer systems and applications"
- A **scenario** is an instance of a **use case** describing a concrete set of actions.
- A **scenario** focuses on understandability.
- We describe a scenario using a template with three fields:
  - The **name**. The name of a scenario is underlined to indicate that it is an instance.
  - The **participating actor**. Actor instances also have underlined names.
  - The **flow of events** describes the sequence of events step by step

Scenarios: FRIEND System Requirement

- In FRIEND field officers, e.g., a police officer or a fireman, have access to a wireless computer that enables them to interact with a dispatcher.
- The dispatcher in turn can visualize the current status of all its resources, such as, police cars or trucks, on a computer screen and dispatch a resource by issuing commands from a workstation.

- **Requirement A.1**
  The accident management system shall allow the field officers to report an accident using a wireless computer.
- **Requirement A.2:**
  The dispatcher in turn can visualize the current status of all its resources, such as, police cars or trucks, on a computer screen and creates an Incident in the database. dispatcher then allocates the proper resources by issuing commands from a workstation.
- **Requirement A.3:**
  The accident management system shall allow the dispatcher to display and view a map of the incident location at any time during the incident recording or allocate resources.

Scenarios: FRIEND Example

Scenario name	warehouseOnFire
Participating actor instances	bob, alice: FieldOfficer john: Dispatcher
Flow of events	1.Bob, driving down main street in his patrol car, notices smoke coming out of a warehouse. His partner, Alice, activates the "Report Emergency" function from her FRIEND laptop. 2.Alice enters the address of the building, a brief description of its location (i.e., northwest corner), and an emergency level. In addition to a fire unit, she requests several paramedic units on the scene given that area appears to be relatively busy. She confirms her input and waits for an acknowledgment. 3.John, the Dispatcher, is alerted to the emergency by a beep of his workstation. He reviews the information submitted by Alice and acknowledges the report. He allocates a fire unit and two paramedic units to the Incident site and sends their estimated arrival time to Alice. 4.Alice receives the acknowledgment .

- In this scenario, a police officer reports a fire and a Dispatcher initiates the incident response.
- Note that this scenario **is concrete**, in the sense that it **does not describe all possible situations** in which a fire can be reported.

Types of Scenarios

- As-is scenarios describe a current situation.
  - For example, the current system is understood by observing users and describing their actions as scenarios.
  - These scenarios can then be validated for correctness and accuracy with the users.
- Visionary scenarios describe a future system.
  - Visionary scenarios can be viewed as an inexpensive prototype
- Evaluation scenarios describe user tasks against which the system is to be evaluated.
- Training scenarios are tutorials used for introducing new users to the system.

How do we find scenarios?

- In RE, developers and users write and refine a series of scenarios in order to gain a shared understanding of what the system should be.
- Don’t rely on interviews and questionnaires alone, insist on task observation if the system already exists
- Ask to speak to the end user, not just to the client.

How do we find scenarios? …

- The following questions can be used for identifying scenarios
  - What are the tasks that the actor wants the system to perform?
  - What information does the actor access? Who creates that data? Can it be modified or removed? By whom?
  - Which external changes does the actor need to inform the system about? How often? When?
  - Which events does the system need to inform the actor about? With what latency?

Other FRIEND Scenarios

- **fenderBender:**
  - A car accident without victims occurs on the highway.
  - Police officers document the incident and manage traffic while the damaged vehicles are towed away.
- **catInATree:**
  - A cat is stuck in a tree.
  - A fire truck is called to retrieve the cat.
  - Because the incident is low priority, the fire truck takes time to arrive at the scene.
  - In the meantime, the impatient cat owner climbs the tree, falls, and breaks a leg, requiring an ambulance to be dispatched.

Finding Scenarios – one more example

1. Consider your watch as a system and set the time 2 minutes ahead.
2. Write down each interaction between you and your watch as a scenario.
3. Record all interactions, including any feedback the watch provides you.
4. The scenario should be detailed enough that an ignorant user can execute the scenario with somebody else's watch.

This scenario could be written using any watch.
We will consider a watch with a digital display and two buttons.

The setWatch2MinutesAhead Scenario

Scenario name	setWatch2MinutesAhead
Participating actor instances	Nasser:WatchOwner
Flow of events	1. Nasser presses both Watch buttons simultaneously. 2. The Watch enters "set time" mode and indicates this by blinking the hour digits. 3. Nasser presses the left button once. 4. The Watch stops blinking the hour digits and starts blinking the minutes. 5. Nasser presses the right button twice. 6. The Watch increments the minutes by two. 7. Nasser presses both buttons simultaneously. 8. The Watch stops blinking.

What's Next

- Next goal after all the scenarios are formulated is to find all the use cases
  - For example: find all the use cases that act as a blueprint for the identified scenario of how to report a fire, therefore,
  - "Report Emergency" in the first paragraph of the scenario is a candidate for a use case.

What’s Next ...

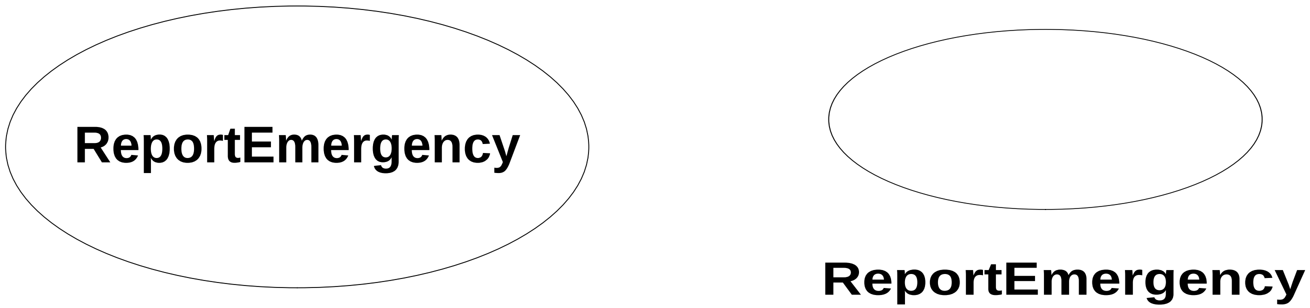
- Describe each of these use cases in more detail
  - Participating actors
  - Describe the entry condition
  - Describe the flow of events
  - Describe the exit condition
  - Describe exceptions
  - Describe nonfunctional requirements
- Functional Modeling

Requirements Elicitation Tasks

- During requirements elicitation, software engineers access many different sources of information.
- These information will be represented in structured forms by applying the following tasks:
  - Identifying actors: software engineers identify the different types of users the future system will support.
  - Identifying scenarios: software engineers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
  - Identifying use cases: software engineers derive from the scenarios a set of use cases that completely represent the future system.
  - Refining use cases: software engineers ensure the completeness of requirements specification by detailing each use case and describing the behaviour of the system in the presence of errors and exceptional conditions.
  - Identifying relationships among use cases: software engineers ensure the consistence of the requirements specification by factoring out the common functionality in the use case model.
  - Identifying nonfunctional requirements: developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.

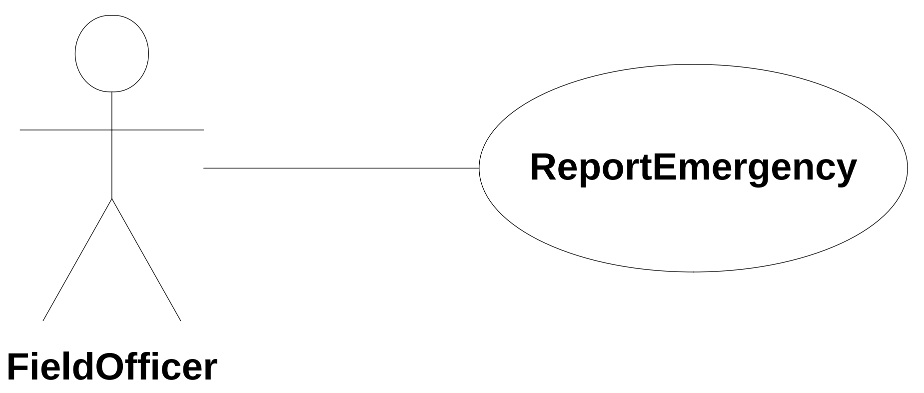
Identifying Use Cases …

- A scenario is an instance of a use case;
  - a use case specifies all possible scenarios for a given piece of functionality.
  - in other word, a use case describes a function provided by the system (the behavior) that yields a **visible result for the actors**.
- A use case is initiated by an actor.
  - A **use case** is something an actor wants the system to do.
  - It is a "case of use" of the system by a specific actor.
- After its initiation, a use case may interact with other actors, as well.
- For example, Requirement A.1 describes one main use of the FRIEND: to report an emergency. The Figure shows how this behavior is captured as a use case.



Communication Lines

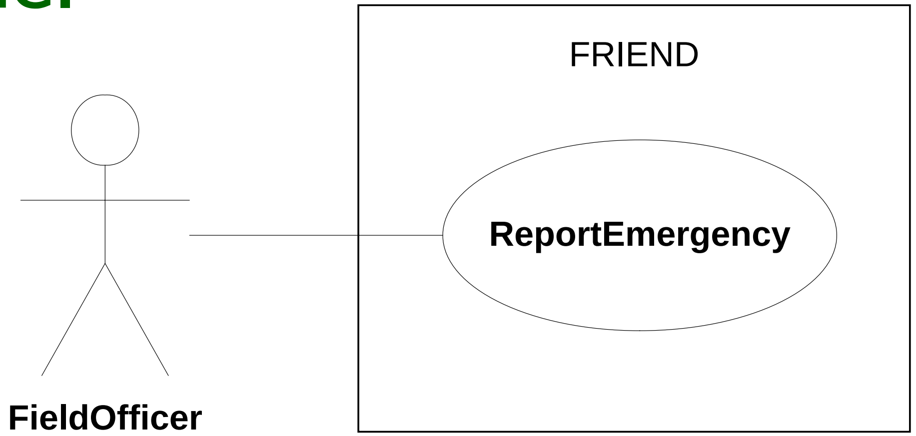
- At this point, we've identified a use case and an actor,
  - but how do we show that the FieldOfficer actor participates in the ReportEmergency use case?
  - The answer is by using **communication lines**.



- Note that, we may have any number of actors involved in a use case. There is no theoretical limit to the number of actors that can participate in a use case.

System Boundaries

- The system boundary is defined by the actors and the use cases
- The system boundary is drawn as a box, labeled with the name of the system, with the actors drawn outside the boundary and the use cases inside.



- You will start use case modeling with only a tentative idea of where the system boundary actually lies. As you find the actors and use cases, the system boundary becomes more and more sharply defined.

Use Case Descriptions

- A diagram showing your use cases and actors does not provide enough detail for your system designers.
  - For example, what steps are involved in the use case?
- The best way to express this important information is in the form of a text-based description. **Every use case should be accompanied by one.**
- There is no UML standard for a use case specification, there are more complex templates in use but it is best to keep use case modeling as simple as possible.
- To describe a use case, we use a template composed of six fields, as shown in the next slide.

ReportEmergency, UC Description, Sample

Use case name	ReportEmergency
Participating actor	FieldOfficer
Entry condition	The FieldOfficer is logged into FRIEND
Flow of events	1. The FieldOfficer activates the "Report Emergency" function of her/his terminal. 2. FRIEND responds by presenting a form to the FieldOfficer. 3. The FieldOfficer fills the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form. 4. FRIEND receives the form and notifies the Dispatcher.
Exit condition	• The FieldOfficer receives an acknowledgment and the selected response from the Dispatcher, OR • The FieldOfficer has received an explanation indicating why the transaction could not be processed.
Quality requirements	• The FieldOfficer's report is acknowledged within 30 seconds. • The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

Use Case Descriptions …

- A use case should describe a complete user transaction
- Use cases should be titled with verb phrases. The title should indicate what the user is trying to perform (e. g., ReportEmergency, OpenIncident).
- The **name** of the use case (unique across the system)
- **Participating actors** are actors interacting with the use case.
- Actors should be titled with noun phrases (e. g., FieldOfficer, Dispatcher).
- The boundary of the system should be clear. Steps performed by the actor and steps performed by the system should be distinguished.
- **Entry conditions (or preconditions)** describe the conditions that need to be satisfied before the use case is initiated.
- **The flow of events** describes the sequence of actions of the use case.
- **Exit conditions (postconditions)** describe the conditions that are satisfied after the completion of the use case.
- **Quality requirements** are not related to the functionality of the system.

ReportEmergency, UC Description, first attempt

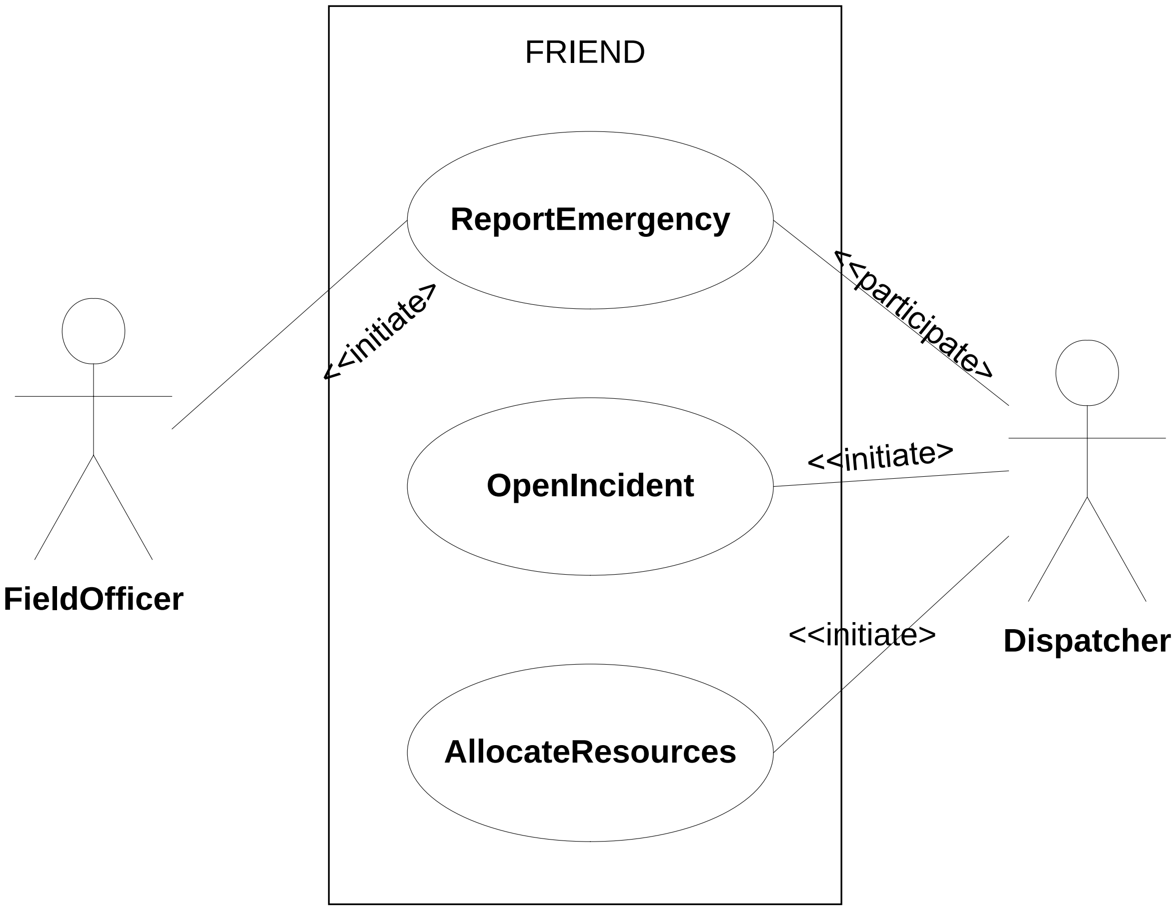
Use case name	ReportEmergency
Participating actor	FieldOfficer
Entry condition	The FieldOfficer is logged into FRIEND
Flow of events	1. The FieldOfficer activates the "Report Emergency" function of her terminal. 2. FRIEND responds by presenting a form to the FieldOfficer. 3. The FieldOfficer fills the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form. 4. FRIEND receives the form and notifies the Dispatcher.
Exit condition	• The FieldOfficer receives an acknowledgment and the selected response from the Dispatcher, OR • The FieldOfficer has received an explanation indicating why the transaction could not be processed.
Quality requirements	• The FieldOfficer's report is acknowledged within 30 seconds. • The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

Remarks

- The use case description has identified a new actor, the **Dispatcher**
- By creating a complete description of the **ReportEmergency** use case, it becomes clear that this actor is **missing**.
- By reviewing this use case model with the client, we found the following requirement.
- Requirement A.2:

  The dispatcher in turn can visualize the current status of all its resources, such as, police cars or trucks, on a computer screen and creates an Incident in the database. The dispatcher then allocates the proper resources by issuing commands from a workstation.
- Let us now revise the UC model and the **ReportEmergency** UC description.

FRIEND UC model has revised



- We started use case modeling with only a tentative idea, and now the system boundary became more defined.

Requirements Elicitation Tasks

- During requirements elicitation, software engineers access many different sources of information.
- These information will be represented in structured forms by applying the following tasks:
  - Identifying actors: software engineers identify the different types of users the future system will support.
  - Identifying scenarios: software engineers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
  - Identifying use cases: software engineers derive from the scenarios a set of use cases that completely represent the future system.
  - Refining use cases: software engineers ensure the completeness of requirements specification by detailing each use case and describing the behaviour of the system in the presence of errors and exceptional conditions.
  - Identifying relationships among use cases: software engineers ensure the consistence of the requirements specification by factoring out the common functionality in the use case model.
  - Identifying nonfunctional requirements: developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.

Refining Use Cases

- The fundamental benefit for the of scenarios and use cases is to define the system requirements that are validated by the user **early** in the development.
  - Because the cost of changing the requirements specification at the design and implementation phases is way higher than in Requirements elicitation phase.
- Although requirements change until late in the development, developers and users should strive to address most requirements issues early.
- Therefore, it is worth doing many changes and much validation during requirements elicitation.
- Keep in mind that, many use cases are revised and rewritten several times, and yet others completely dropped.

ReportEmergency, UC Description, second attempt

Use case name	ReportEmergency
Participating actor	Initiated by FieldOfficer Communicated with Dispatcher
Entry condition	The FieldOfficer is logged into FRIEND
Flow of events	1.The FieldOfficer activates the “Report Emergency” function of her terminal. 2.FRIEND responds by presenting a form to the FieldOfficer. 3.The FieldOfficer fills the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form. 4.FRIEND receives the form and notifies the Dispatcher. 5.The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the report. 6.FRIEND displays the acknowledgement and selected response to the FieldOfficer.
Exit condition	same
Quality req.	same

Use Case Relationships

- Consider the following other requirement.
- Requirement A.3:

  The accident management system shall allow the dispatcher to display and view a map in order to determine the incident location at any time during the incident recording or allocate resources.

- Let us know consider the UC description for both **OpenIncident** UC and **AllocateResources** UC.

OpenIncident Use Case

Use case name	OpenIncident
Participating actor	Initiated by the Dispatcher
Entry condition	• The Dispatcher is alerted to the emergency by a beep of his workstation. • An EmergencyReport is received.
Flow of events	1.The Dispatcher activates the "Open Incident" function of her station. 2.FRIEND responds by presenting the incident form to the Dispatcher. 3.All the information contained in the FieldOfficer's form is automatically included in the incident form. 4.The Dispatcher reviews the information submitted by the FieldOfficer. 5.The Dispatcher display and view the map to identify the location of an incident. 6.The Dispatcher complete the form and submit to creates an Incident in the database. 7.FRIEND responds by acknowledges the Dispatcher an incident has successfully created.
Exit condition	• The Incident is created in the database. • The Dispatcher is acknowledged that the incident has successfully created.
Quality requirements	• When a map is displayed, the system zoom in the map so that location of the current Incident is visible to the Dispatcher.

AllocateResources use case

Use case name	AllocateResources
Participating actor	Initiated by the Dispatcher
Entry condition	• The Dispatcher is acknowledged that the incident has successfully created.
Flow of events	1.The Dispatcher activates the “Allocate Resources” function of her station. 2.FRIEND responds by presenting an incident form to the Dispatcher. 3.All the information recorded in “Open Incident” function is automatically included in the incident form. 4.The Dispatcher display and view the map to identify the location of an incident. 5.Based on the physical location, the Dispatcher allocate the proper resources such as a Police car or a Fire truck and then submits the form. 6.FRIEND receives the form and notifies the Dispatcher.
Exit condition	• Additional Resources are assigned to the Incident • Resources receives notice about their new assignment. • FieldOfficer in charge of the Incident receives notice about the new Resources.
Quality requirements	• When a map is displayed, the system zoom in the map so that location of the current Incident is visible to the Dispatcher.

Writing Use cases – one more example

Consider the **setWatch2MinutesAhead** scenario

1. Identify the actor of the scenario.
2. Next, write the corresponding use case SetTime. Include all cases, and include setting the time forward, backward, setting hours, minutes, and seconds.
3. The level of detail of the use case should be the same as the level of detail of the UC in the previous example.

Use case name	SetTime
Participating actor	Initiated by WatchOwner
Entry condition	The Watch is in “read time” mode
Flow of events	1. The WatchOwner presses both Watch buttons simultaneously. 2. The Watch enters “set hour” mode and indicates this by blinking the hour digits. 3. In the “set hour” mode, the WatchOwner can advance the hour digits by pressing the right button. Each time the right button is pressed, the hours are incremented by one. If the hours reach 23 and the right button is pressed, the hours are reset to 0. The date is not changed. 4. At any time in the “set hour” mode, the WatchOwner can switch to the “set minutes” mode by selecting the left button. 5. To indicate this, the Watch stops blinking the hour digits and starts blinking the minute digits. 6. In the “set minutes” mode, the WatchOwner can advance the minute digits by pressing the right button. Each time the right button is pressed, the minutes are incremented by one. If the minutes reach 59 and the right button is pressed, the minutes are reset to 0. The hours are not changed.

Flow of events	7. At any time in the “set minutes” mode, the WatchOwner can switch to the “set seconds” mode by selecting the left button.
	    8. To indicate this, the Watch stops blinking the minute digits and starts blinking the second digits.
	9. In the “set seconds” mode, the WatchOwner can advance the second digits by pressing the right button. Each time the right button is pressed, the seconds are incremented by one. If the seconds reach 59 and the right button is pressed, the seconds are rest to 0. The hours and the minute digits are not changed.
	10. At any time in the “set seconds” mode, the WatchOwner can switch to the “set hours” mode by pressing the left button.
	    11. To indicate this, the Watch stops blinking the second digits and starts blinking the hour digits.
	12. At any time in the “set hours,” “set minutes,” and “set seconds” mode, the WatchOwner can return to “read time” mode by pressing both Watch buttons simultaneously.

Exit condition	• The Watch is in “read time” mode with the new time set.
Quality req.	• none.

Writing Use cases – similar example

1. Assume the watch system we described in the previous also supports an alarm feature.
2. Describe setting the alarm time as a self-contained use case named **SetAlarmTime**.
3. The level of detail of the use case should be the same as the level of detail of the previous example.

Use case name	SetAlarmTime
Participating act	Initiated by WatchOwner
Entry condition	The Watch is in "read time" mode
Flow of events	1.The WatchOwner presses both Watch buttons simultaneously. 2.The Watch enters "set hour" mode and indicates this by blinking the hour digits. 3.<u>The WatchOwner continues to press both buttons.</u> 4.<u>After one second, a small alarm bell symbol appears on the display and blinks, indicating that the hours of the alarm are being set (as opposed to the normal time).</u> 5.In the "set hour" mode, the WatchOwner can advance the hour digits by pressing the right button. Each time the right button is pressed, the hours are incremented by one. If the hours reach 23 and the right button is pressed, the hours are reset to 0. The date is not changed. 6.At any time in the "set hour" mode, the WatchOwner can switch to the "set minutes" mode by selecting the left button. 7.To indicate this, the Watch stops blinking the hour digits and starts blinking the minute digits. 8.In the "set minutes" mode, the WatchOwner can advance the minute digits by pressing the right button. Each time the right button is pressed, the minutes are incremented by one. If the minutes reach 59 and the right button is pressed, the minutes are reset to 0. The hours are not changed.

Flow of events	9. At any time in the "set minutes" mode, the WatchOwner can switch to the "set seconds" mode by selecting the left button. 10. To indicate this, the Watch stops blinking the minute digits and starts blinking the second digits. 11. In the "set seconds" mode, the WatchOwner can advance the second digits by pressing the right button. Each time the right button is pressed, the seconds are incremented by one. If the seconds reach 59 and the right button is pressed, the seconds are rest to 0. The hours and the minute digits are not changed. 12. At any time in the "set seconds" mode, the WatchOwner can switch to the "set hours" mode by pressing the left button. 13. To indicate this, the Watch stops blinking the second digits and starts blinking the hour digits. 14. At any time in the "set hours," "set minutes," and "set seconds" mode, the WatchOwner can return to "read time" mode by pressing both Watch buttons simultaneously.
Exit condition	• The Watch is in "read time" mode with the new time set.
Quality reqs	• none.

- The rationale behind this similarity is that any user interface should provide similar interfaces for addressing similar functionality (i.e., user interface consistency, MSWord, MSExcel, MSPPT)

Requirements Elicitation Tasks

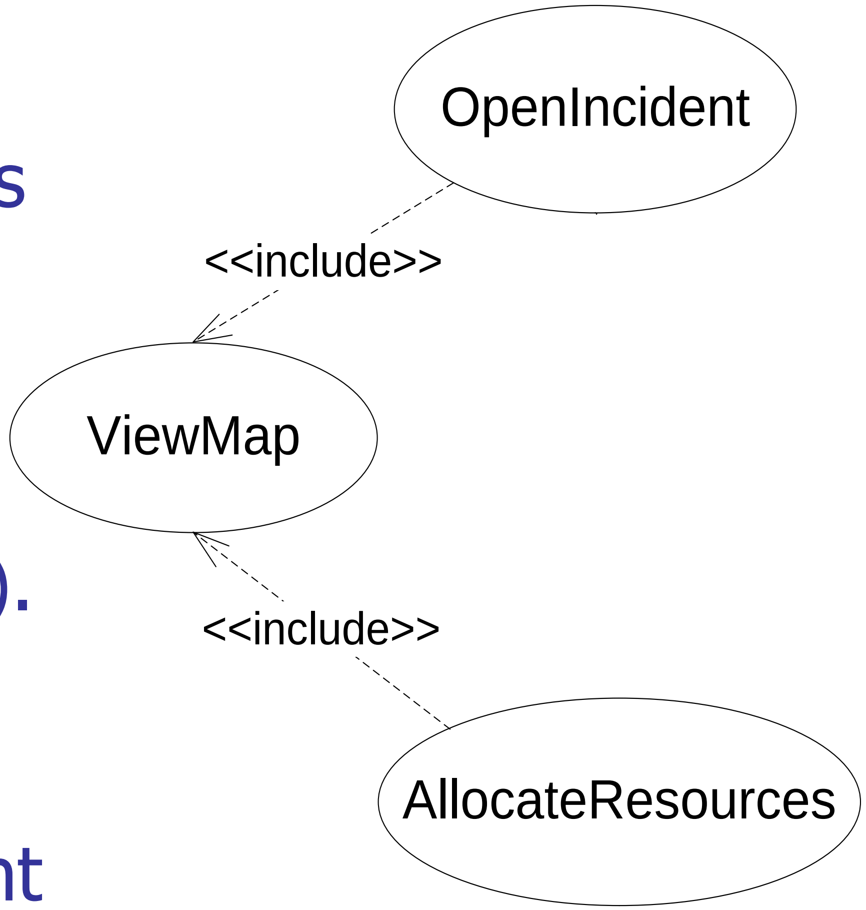
- During requirements elicitation, software engineers access many different sources of information.
- These information will be represented in structured forms by applying the following tasks:
  - Identifying actors: software engineers identify the different types of users the future system will support.
  - Identifying scenarios: software engineers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
  - Identifying use cases: software engineers derive from the scenarios a set of use cases that completely represent the future system.
  - Refining use cases: software engineers ensure the completeness of requirements specification by detailing each use case and describing the behaviour of the system in the presence of errors and exceptional conditions.
  - Identifying relationships among use cases: software engineers ensure the consistence of the requirements specification by factoring out the common functionality in the use case model.
  - Identifying nonfunctional requirements: developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.

Use Case Relationships …

- When you examine the previous **OpenIncident** UC and **AllocateResources** UC, you will notice that there is some repetition between steps in these two different use cases.
  - For example, step 5 ◄ in **OpenIncident** UC and step 4 ◄ in **AllocateResources** UC are the same and representing an identical behaviour.
- This repetitive behavior is **best separated** and captured within a totally **new use case**.
- This new use case can then be **reused** by the **OpenIncident** UC and **AllocateResources** UC using the <<include>> relationship.

Include relationships between use cases

- Redundancies among use cases can be factored out using include relationships.
- For example, assume that a Dispatcher needs to consult the city map
  - when opening an incident (e. g., to assess which areas are at risk during a fire) and
  - when allocating resources (e. g., to find which resources are closest to the incident).
- In this case, the ViewMap use case describes the flow of events required when viewing the city map and is used by both the OpenIncident and the AllocateResources use cases.



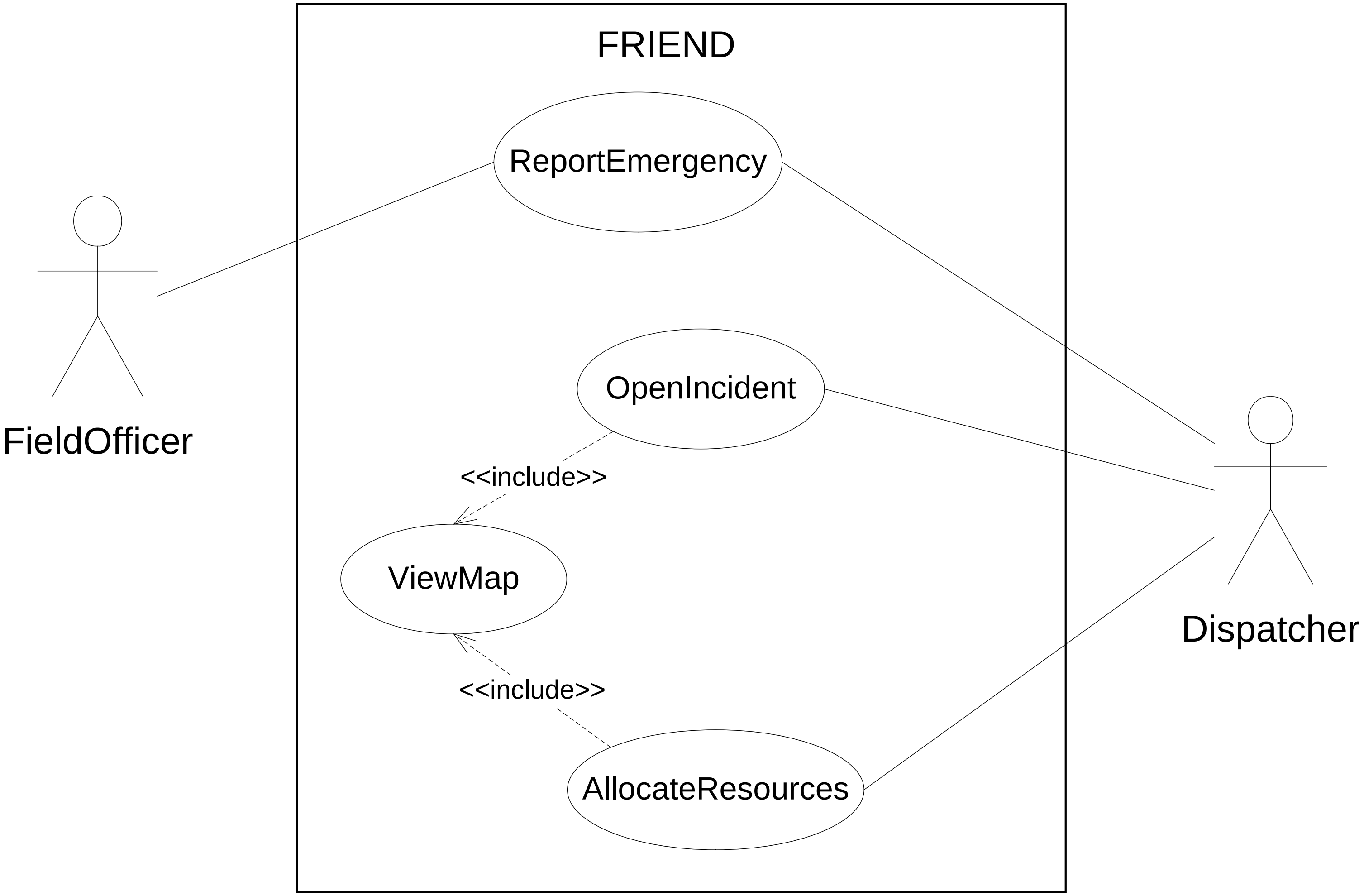
OpenIncident use case

Use case name	OpenIncident
Participating actor	Initiated by the Dispatcher
Entry condition	• The Dispatcher is alerted to the emergency by a beep of his workstation. • An EmergencyReport is received.
Flow of events	1.The Dispatcher activates the "Open Incident" function of her station. 2.FRIEND responds by presenting the incident form to the Dispatcher. 3.All the information contained in the FieldOfficer's form is automatically included in the incident form. 4.The Dispatcher reviews the information submitted by the FieldOfficer. 5.The Dispatcher display and view the map to identify the location of an incident. 6.The Dispatcher complete the form and submit to creates an Incident in the database. 7.FRIEND responds by acknowledges the Dispatcher an incident has successfully created.
Exit condition	• The Incident is created in the database. • The Dispatcher is acknowledged that the incident has successfully created.
Quality reqs	• When a map is displayed, the system zoom in the map so that location of the current Incident is visible to the Dispatcher.

AllocateResources use case

Use case name	AllocateResources
Participating actor	Initiated by the Dispatcher
Entry condition	• The Dispatcher is acknowledged that the incident has successfully created.
Flow of events	1.The Dispatcher activates the "Allocate Resources" function of her station. 2.FRIEND responds by presenting an incident form to the Dispatcher. 3.All the information recorded in "Open Incident" function is automatically included in the incident form. 4.The Dispatcher display and view the map to identify the location of an incident. 5.Based on the physical location, the Dispatcher allocate the proper resources such as a Police car or a Fire truck and then submits the form. 6.FRIEND receives the form and notifies the Dispatcher.
Exit condition	• Additional Resources are assigned to the Incident • Resources receives notice about their new assignment. • FieldOfficer in charge of the Incident receives notice about the new Resources
Quality requirements	• When a map is displayed, the system zoom in the map so that location of the current Incident is visible to the Dispatcher.

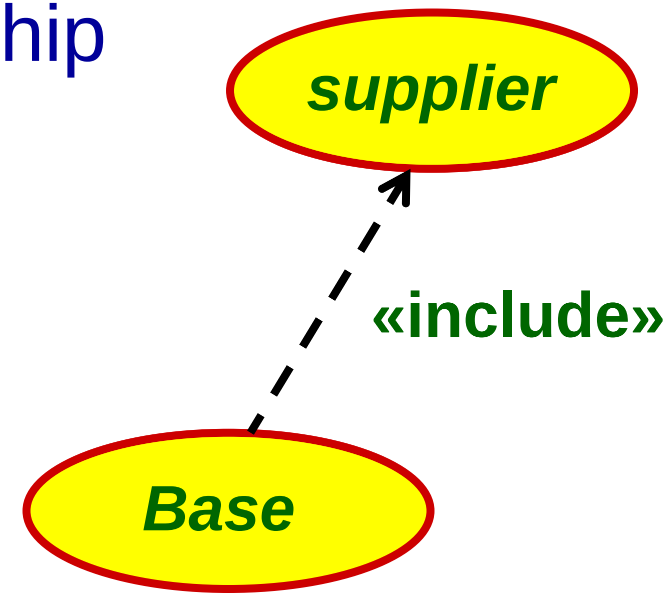
FRIEND UC model has revised

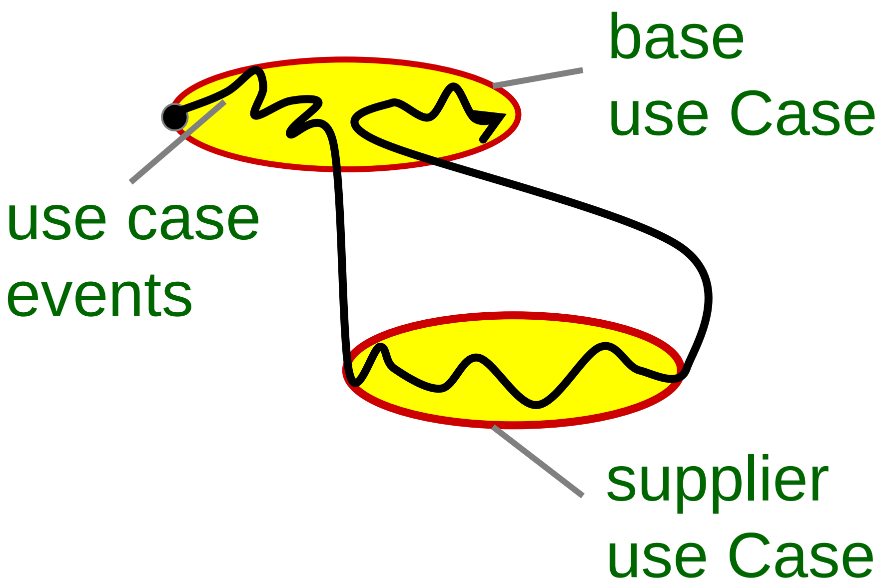


- We started use case modeling with only a tentative idea, and now the system boundary became more defined.

What is the «include» Relationship?

- Two use cases are related by an include relationship if one of them (the base) includes the second one (the supplier) in its flow of events.
- In use case diagrams, **include relationships** are depicted by a dashed open arrow originating from the including use case, and are labelled with the string <<include>>.





- The behavior defined in the inclusion use case is explicitly included into the base use case.
- The syntax for «include» is a bit like a function call.

The semantics of «include»

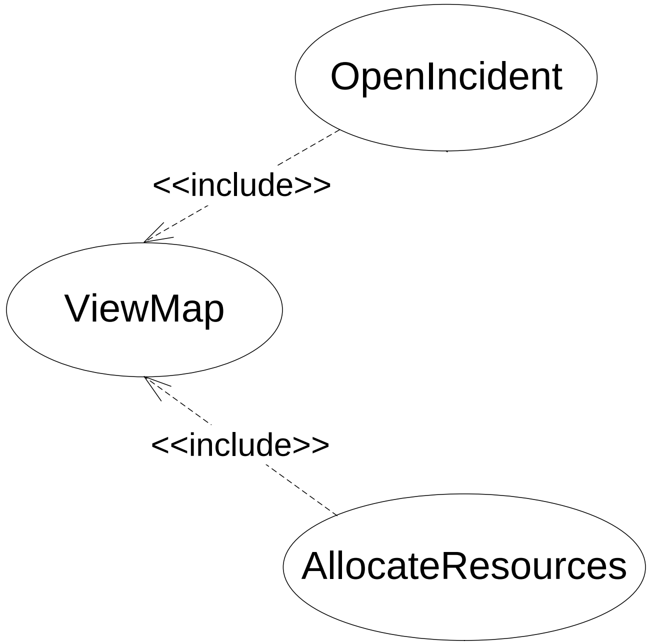
- The base use case executes until the point of inclusion is reached, then execution passes over to the supplier use case.
- When the supplier finishes, control returns to the base again.
- The base use case is not complete without all of its supplier use cases
- The supplier use cases form integral parts of the base use case. However, the supplier use cases may or may not be complete.
- If a supplier use case is *not* complete, then it just contains a partial flow of events that will only make sense when it is included into a suitable base.
  - In this case we call it “a behavior fragment” and it can’t be triggered directly by actors.
- If the supplier use cases are complete in themselves, then they act just like normal use cases.

Include relationships in the UC description

- We represent include relationships in the textual description of the use case with one of two ways.
  - If the included use case can be included at any point in the flow of events (e. g., the ViewMap use case), we indicate the inclusion in the Quality requirements section of the use case (see next slide).
  - If the included use case is invoked during an event, we indicate the inclusion in the flow of events.

Textual representation of **Include** relationships

Use case name	AllocateResources
Participating actor	Initiated by the Dispatcher
Entry condition	• The Dispatcher is acknowledged that the incident has successfully created.
Flow of events	. . .
Exit condition	• Additional Resources are assigned to the Incident • Resources receives notice about their new assignment. • FieldOfficer in charge of the Incident receives notice about the new Resources.
Quality requirements	• At any point during the flow of events, this use case can **include** the ViewMap use case. The ViewMap use case is initiated when the Dispatcher invokes the map function. When invoked within this use case, the system scrolls the map so that location of the current Incident is visible to the Dispatcher.



Why use an «include» Relationship?

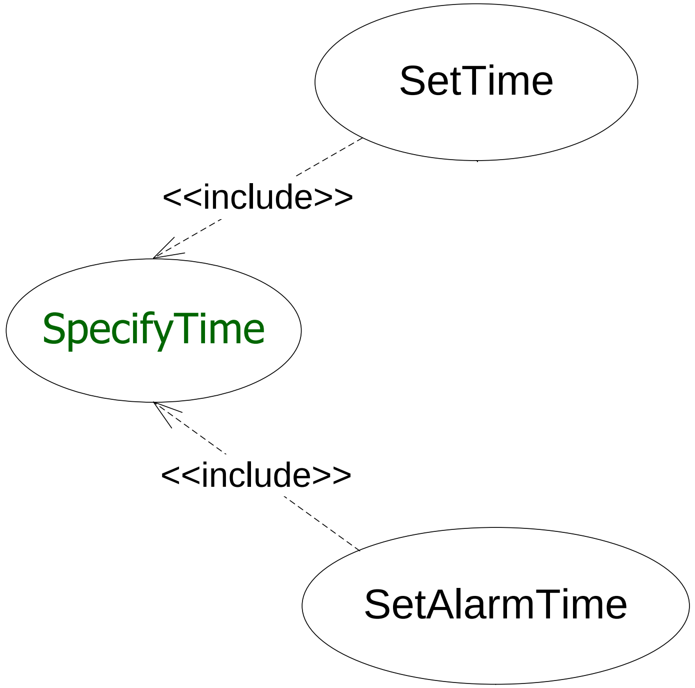
- Factor out behavior common to two or more use cases.
  - Avoid describing same behavior multiple times.
  - Assure common behavior remains consistent.

- Factor out and encapsulate behavior from a base use case.
  - Simplify complex flow of events.
  - Factor out behavior not part of primary purpose.

Back to SetTime & SetAlarmTime examples

1. Examine the **SetTime** and **SetAlarmTime** use cases we wrote in Slides 60 to 65.
2. Eliminate any redundancy by using an include relationship.

- We will re-write the two use cases SetTime, SetAlarmTime, and write a new use case (called SpecifyTime) made out of the common parts of the original SetTime and SetAlarmTime.
- The essential point is to factor out as much common functionality as possible into the third use case.



SetTime use case

UC name	SetTime
Participating actor	Initiated by WatchOwner
Entry condition	The Watch is in “read time” mode
Flow of events	1.The WatchOwner presses both Watch buttons simultaneously. 2.The Watch enters the “sethour” mode and indicates this by blinking the hour digits. 3.The SpecifyTime use case is included here. A the end of the SpecifyTime use case, the WatchOwner has specified a time. 4.The WatchOwner presses both buttons simultaneously to return to the “read time” mode. The new time is set.
Exit condition	• The Watch is in “read time” mode with the new time set.
Quality reqs	• none.

SetAlarmTime use case

Use case name	SetAlarmTime
Participating actor	Initiated by WatchOwner
Entry condition	The Watch is in “read time” mode
Flow of events	1.The WatchOwner presses both Watch buttons simultaneously. 2.The Watch enters the “sethour” mode and indicates this by blinking the hour digits. 3.The WatchOwner continues to press both buttons. 4.After one second, a small alarm bell symbol appears on the display and blinks, indicating that the hours of the alarm are being set (as opposed to the normal time). 5.The SpecifyTime use case is included here. A the end of the SpecifyTime use case, the WatchOwner has specified a time for the alarm. 6.The WatchOwner presses both buttons simultaneously to return to the “read time” mode. The new alarm time is set.
Exit condition	• The Watch is in “read time” mode with the new time set.
Quality reqs	• none.

SpecifyTime use case

Use case name	SpecifyTime
Participating act	Initiated by WatchOwner
Entry condition	The Watch is in “set alarm” or “set time” mode. The hour digits are blinking.
Flow of events	1. In the “set hour” mode, the WatchOwner can advance the hour digits by pressing the right button. Each time the right button is pressed, the hours are incremented by one. If the hours reach 23 and the right button is pressed, the hours are reset to 0. The date is not changed. 2. At any time in the “set hour” mode, the WatchOwner can switch to the “set minutes” mode by selecting the left button. To indicate this, the Watch stops blinking the hour digits and starts blinking the minute digits. 3. In the “set minutes” mode, the WatchOwner can advance the minute digits by pressing the right button. Each time the right button is pressed, the minutes are incremented by one. If the minutes reach 59 and the right button is pressed, the minutes are reset to 0. The hours are not changed. 4. At any time in the “set minutes” mode, the WatchOwner can switch to the “set seconds” mode by selecting the left button. To indicate this, the Watch stops blinking the minute digits and starts blinking the second digits.

SpecifyTime use case …

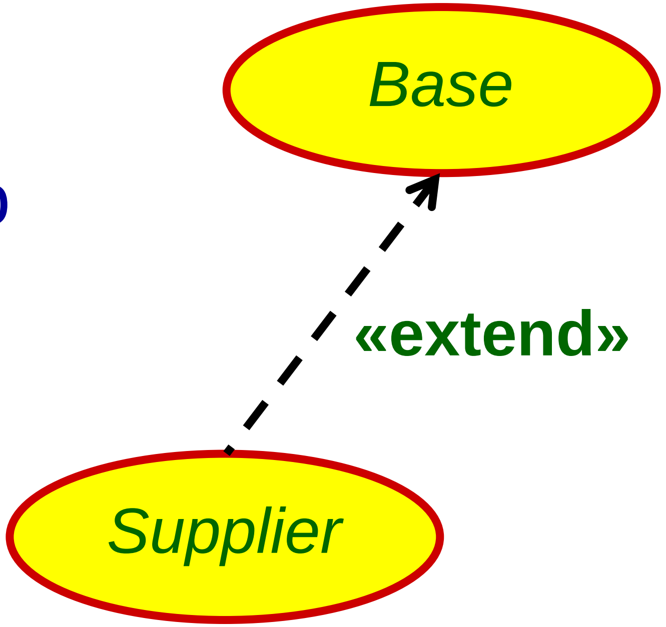
Flow of events	5. In the "set seconds" mode, the WatchOwner can advance the second digits by pressing the right button. Each time the right button is pressed, the seconds are incremented by one. If the seconds reach 59 and the right button is pressed, the seconds are rest to 0. The hours and the minute digits are not changed. 6. At any time in the "set seconds" mode, the WatchOwner can switch to the "set hours" mode by pressing the left button. To indicate this, the Watch stops blinking the second digits and starts blinking the hour digits. 7. At any time in the "set hours," "set minutes," and "set seconds" mode, the WatchOwner can return to "read time" mode by pressing both Watch buttons simultaneously. The digits stop blinking and the alarm bell stays on and stops blinking.
Exit condition	• The Watch is in "read time" mode with the new time set.
Quality reqs	• none.

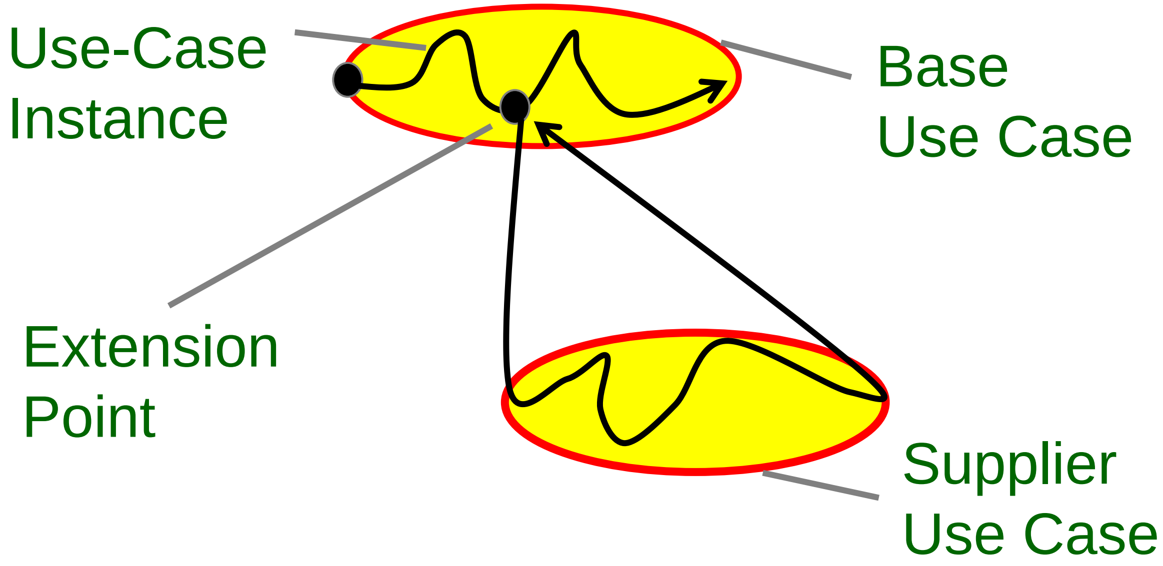
The «extend» relationships between UCs

- In some situations we need to represent an **exceptional behavior** of the use case flow.
- For example, assume that the network connection between the Dispatcher and the FieldOfficer can be interrupted at any time. (e.g., if the FieldOfficer enters a tunnel).
- If we start to describe the set of events taken by the system and the actors while the connection is lost, our use cases become more complicated.
- Solution, **Extend relationships** are an alternate means for reducing complexity in the use case model.

What is an «extend» Relationship?

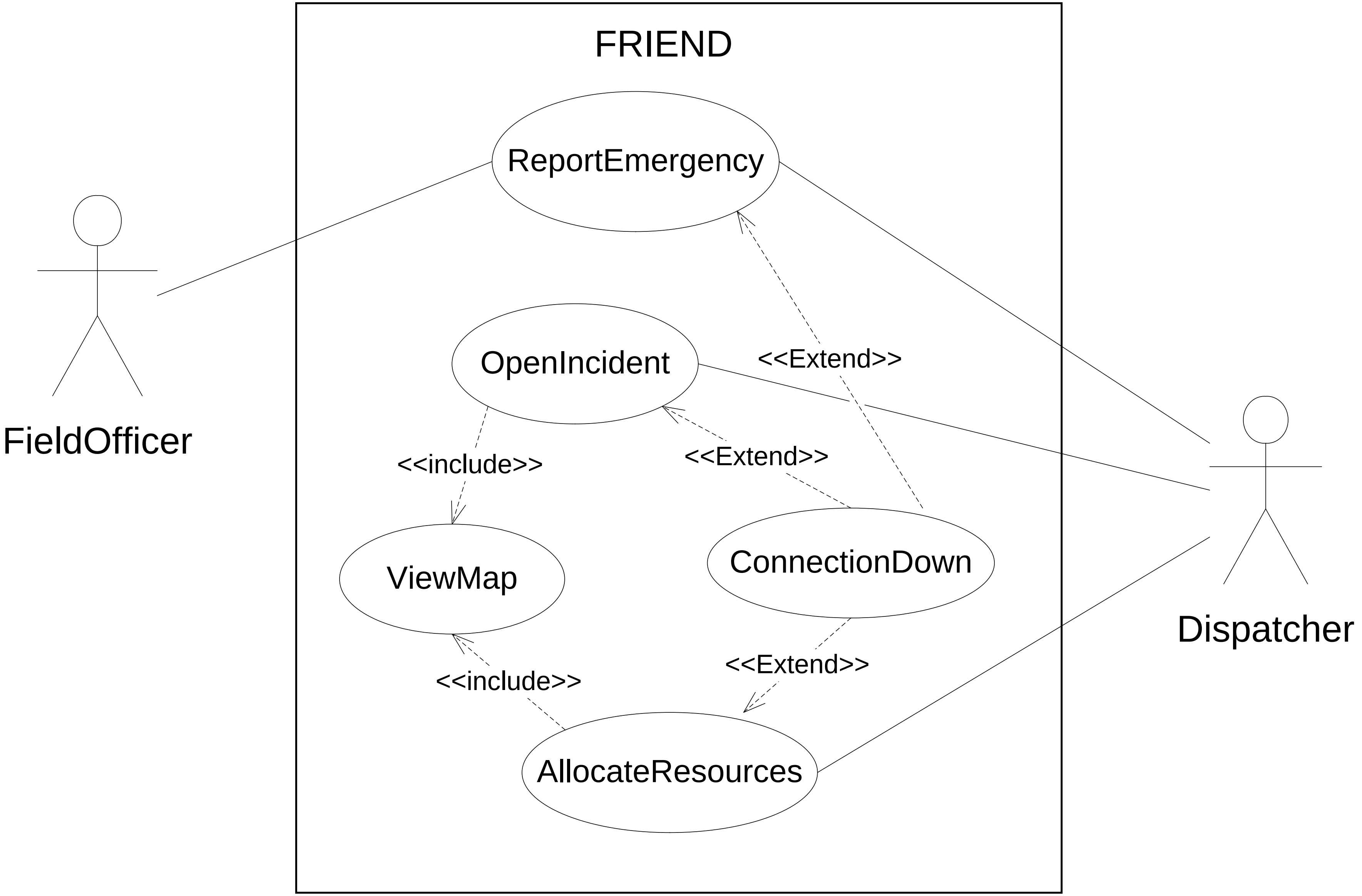
- The <<extend>> relationships are drown like the <<include>> relationships, but
  - The arrow direction is from the supplier use case to the base use case
  - Labelled by a string <<extend>>
- An extend relationship indicates that an instance of the base use case may include (**under certain conditions**) the behavior specified by the supplier use case.





- Supplier UC is executed when the extension point in the Base is reached and the extending **condition is true.**
- Supplier UC is inserted into base use case at **named** extension point.

FRIEND UC model has revised

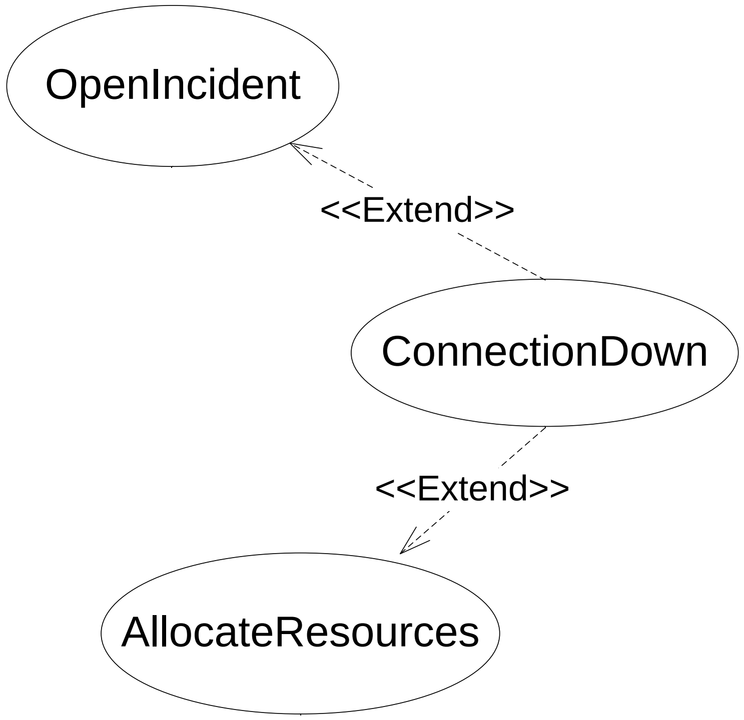


- We started use case modeling with only a tentative idea, and now the system boundary became more defined.

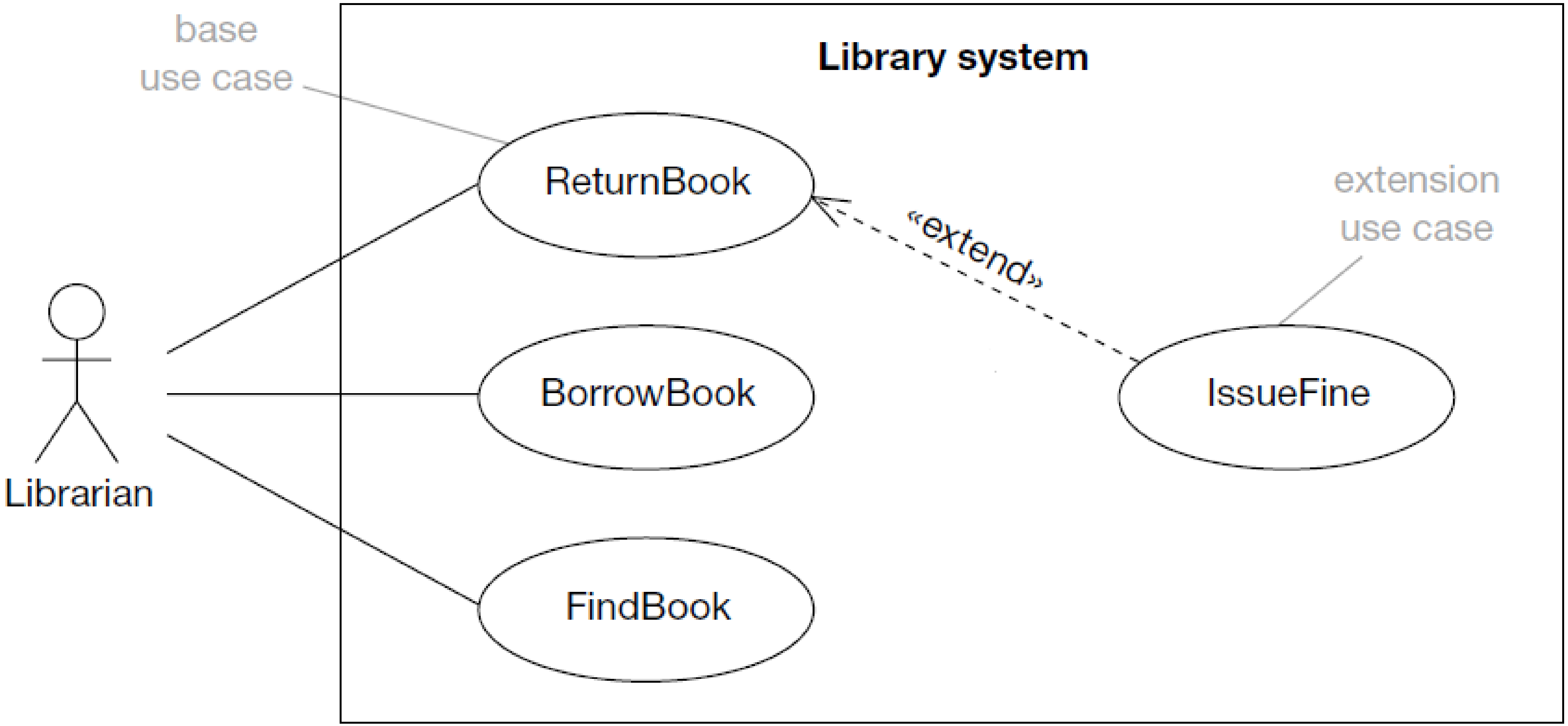
Textual representation of extend relationships

- In the textual representation of a use case, we represent extend relationships as entry conditions of the extending use case.

Use case name	ConnectionDown
Participating actor	Communicates with FieldOfficer and Dispatcher.
Entry condition	• This use case **extends** the OpenIncident and the AllocateResources use cases. It is initiated by the system whenever the network connection between the FieldOfficer and Dispatcher is lost.
Flow of events	. . .
Exit condition	. . .



The «extend», one more example



- The librarian can perfectly complete the function "ReturnBook" without the need to execute the extension "IssueFine"

The semantics of «extend»

- The base use case provides a set of extension points which are hooks where new behavior may be added.
- The supplier use case provides a set of behavior fragments that can be inserted into the base use case at these hooks.
- The base use case does not know anything about the supplier use cases – it just provides hooks for them. In fact, the base use case is perfectly complete without its extensions.
  - This is very different to «include» where the base use cases were incomplete without their included use cases.
- The base use case flow is completely independent of the extension points.
- Supplier use cases are generally not complete use cases and therefore can't usually be instantiated by an actor.

Why use an «extend» relationship?

- Factor out optional or exceptional behavior.
  - Executed only under certain conditions.
  - Simplifies flow of events in base use case.

- In fact you should be able to produce a pretty complete first cut Use Case Model without any «extend» relationships and we would advise you to do this. The extending Use Cases represent additional behaviors, without which the model is consistent in itself.

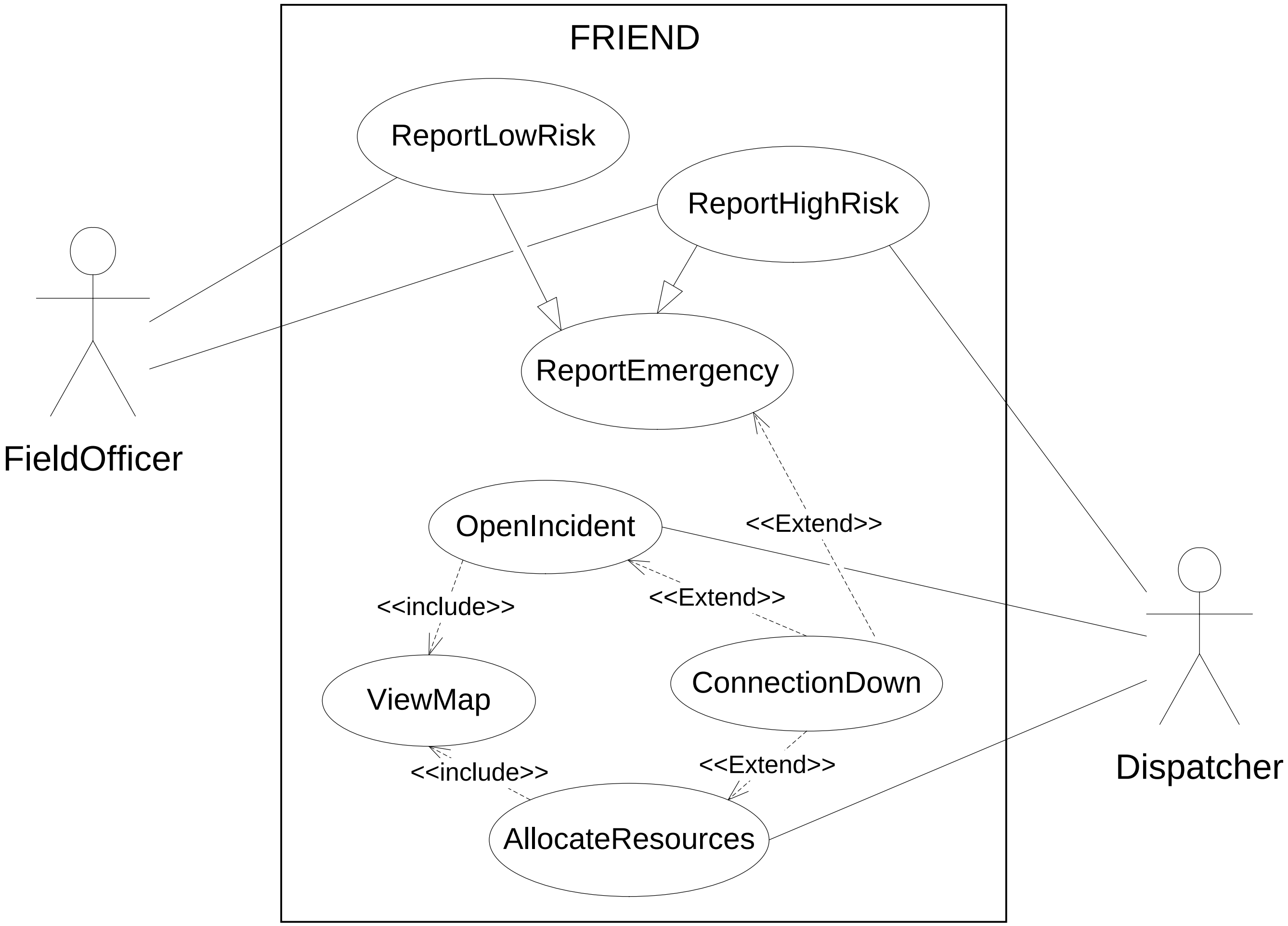
The difference between the include and extend

- The difference between the include and extend relationships is the location of the dependency.
- Assume that we add several new use cases for the actor Dispatcher, such as UpdateIncident and ReallocateResources.
- If we modeled the ConnectionDown use case with include relationships, the authors of UpdateIncident and ReallocateResources use cases need to know about and include the ConnectionDown use case.
- If we used extend relationships instead, only the ConnectionDown use case needs to be modified to extend the additional use cases.
- In general exception cases (such as help, errors, and other unexpected conditions) are modeled with extend relationships.
- Use cases that describe behavior commonly shared by a limited set of use cases are modeled with include relationships.

Inheritance Relationships

- Sometimes you'll come across a use case whose behavior can be applied to several different cases, **but with small changes**.
- For example, currently our ReportEmergency UC deals with all incidents equally. Assume that the requirements have changed such that:
  - The FieldOfficer shall distinguish between two types of incidents, low risk incidents and high risk incidents. Only the high risk incidents needs additional resources, such as fire truck. The low risk incident can be completely managed by the FieldOfficer.
  - In this case, both incident are emergency cases and need to be reported, and each of them has a special behaviour as well.
- **This is where use case inheritance relationships comes in.**

FRIEND UC Model has revised



- We started use case modeling with only a tentative idea, and now the system boundary became more defined.

UML Use Case – the last word

- Generally, stakeholders can easily understand actors and use cases with just a little training
- Stakeholders find actor generalization more difficult to grasp
- Heavy use of «include» can make use case models harder to understand – stakeholders and developers have to look at more than one use case to get the complete picture
- Stakeholders have great difficulty with «extend» – this can be true even with careful explanation
- A surprising number of developers misunderstand the semantics of «extend»
- Use case generalization should be avoided unless abstract (rather than concrete) parent use cases are used

Requirements Elicitation Tasks

- During requirements elicitation, software engineers access many different sources of information.
- These information will be represented in structured forms by applying the following tasks:
  - Identifying actors: software engineers identify the different types of users the future system will support.
  - Identifying scenarios: software engineers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
  - Identifying use cases: software engineers derive from the scenarios a set of use cases that completely represent the future system.
  - Refining use cases: software engineers ensure the completeness of requirements specification by detailing each use case and describing the behaviour of the system in the presence of errors and exceptional conditions.
  - Identifying relationships among use cases: software engineers ensure the consistence of the requirements specification by factoring out the common functionality in the use case model.
  - Identifying nonfunctional requirements: developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.

Identifying nonfunctional requirements

- Nonfunctional requirements describe aspects of the system that are not directly related to its functional behavior.
- Nonfunctional requirements span a number of issues,
  - from user interface look and feel
    - to response time requirements
      - to security issues.
- Nonfunctional requirements are defined at the same time as functional requirements because they have as much impact on the development and cost of the system.
- The following table shows example questions help identify nonfunctional requirements

Heuristics for finding Nonfunctional Requirements

Category	**Example questions**
Usability	○ What is the level of expertise of the user? ○ What user interface standards are familiar to the user? ○ What documentation should be provided to the user?
Reliability (including robustness, safety, and security)	○ How reliable, available, and robust should the system be? ○ Is restarting the system acceptable in the event of a failure? ○ How much data can the system loose? ○ How should the system handle exceptions? ○ Are there safety requirements of the system? ○ Are there security requirements of the system?
Performance	○ How responsive should the system be? ○ Are any user tasks time critical? ○ How many concurrent users should it support? ○ How large is a typical data store for comparable systems? ○ What is the worse latency that is acceptable to users?

Heuristics for finding Nonfunctional Requirements

Category	Example questions
Supportability (including maintainability and portability)	○ What are the foreseen extensions to the system? ○ Who maintains the system? ○ Are there plans to port the system to different software or hardware environments?
Implementation	○ Are there constraints on the hardware platform? ○ Are constraints imposed by the maintenance team? ○ Are constraints imposed by the testing team?
Interface	○ Should the system interact with any existing systems? ○ How are data exported/ imported into the system? ○ What standards in use by the client should be supported by the system?
Operation	○ Who manages the running system?
Packaging	○ Who installs the system? ○ How many installations are foreseen? ○ Are there time constraints on the installation?
Legal	○ How should the system be licensed? ○ Are any liability issues associated with system failures? ○ Are any royalties or licensing fees incurred by using specific algorithms or components?

Requirements Elicitation Methods

Methods to Elicit Requirements

- Bridging the gap between end user and developer:
  - **Interviewing customers and domain experts**
  - **Questionnaires**
  - **Task Analysis (Observation)**
  - **Study of documents and software systems**
  - **JAD, Joint Application Development**

Interviews - Five Basic Steps

1. Selecting interviewees
    - Often good to get different perspectives
        - Managers, Users, Ideally all key stakeholders
2. Designing interview questions
    - Closed-Ended Questions
        - How many telephone orders are received per day?
        - How do customers place orders?
    - Open-Ended Questions
        - What do you think about the current system?
        - What are some of the problems you face on a daily basis?
3. Preparing for the interview
    - Prepare general interview plan: list of question and anticipated answers and follow-ups
    - Inform the interviewee about the reason for interview and the areas of discussion

Interviews - Five Basic Steps …

4. Conducting the interview
   - Appear professional and unbiased
   - Record all information
   - Give interviewee time to ask questions
   - Be sure to thank the interviewee
   - End on time
5. Post-interview follow-up
   - Prepare interview notes
   - Prepare interview report
   - Look for gaps and new questions

Questionnaires

- In addition to interviews
- Questionnaire is a passive technique
  - advantage – time to consider the answers
  - disadvantage – no possibility to clarify questions and answers
- Close-ended questions
  - Multiple-choice questions
    - additional comments may be allowed
  - Rating questions (e.g. strongly agree, agree, ...)
    - when seeking opinions
  - Ranking questions
    - ranked with numbers, percent values, etc.

Task Analysis (Observation)

- In addition to interviews (and possibly questionnaires)
- When the user cannot convey sufficient information and/or has only fragmented knowledge
- Three forms
  - Passive
    - no interruption or direct involvement
    - video camera may be used
  - Active
  - Explanatory
    - explaining what is done when observed
- Should be carried for a long time, at different times and at different workloads
- Note that, people tend to behave differently when watched

Study of Documents and Software Systems

- Provides clues about existing “as-is” system
- Typical documents
  - Organizational documents
    - including procedures, policies, descriptions, plans, charts, internal and external correspondence
  - System forms and reports (if prior computer system exists)
    - record of change requests (defects and enhancements)
- Domain knowledge requirements
  - domain journals and reference books
  - using Internet searches

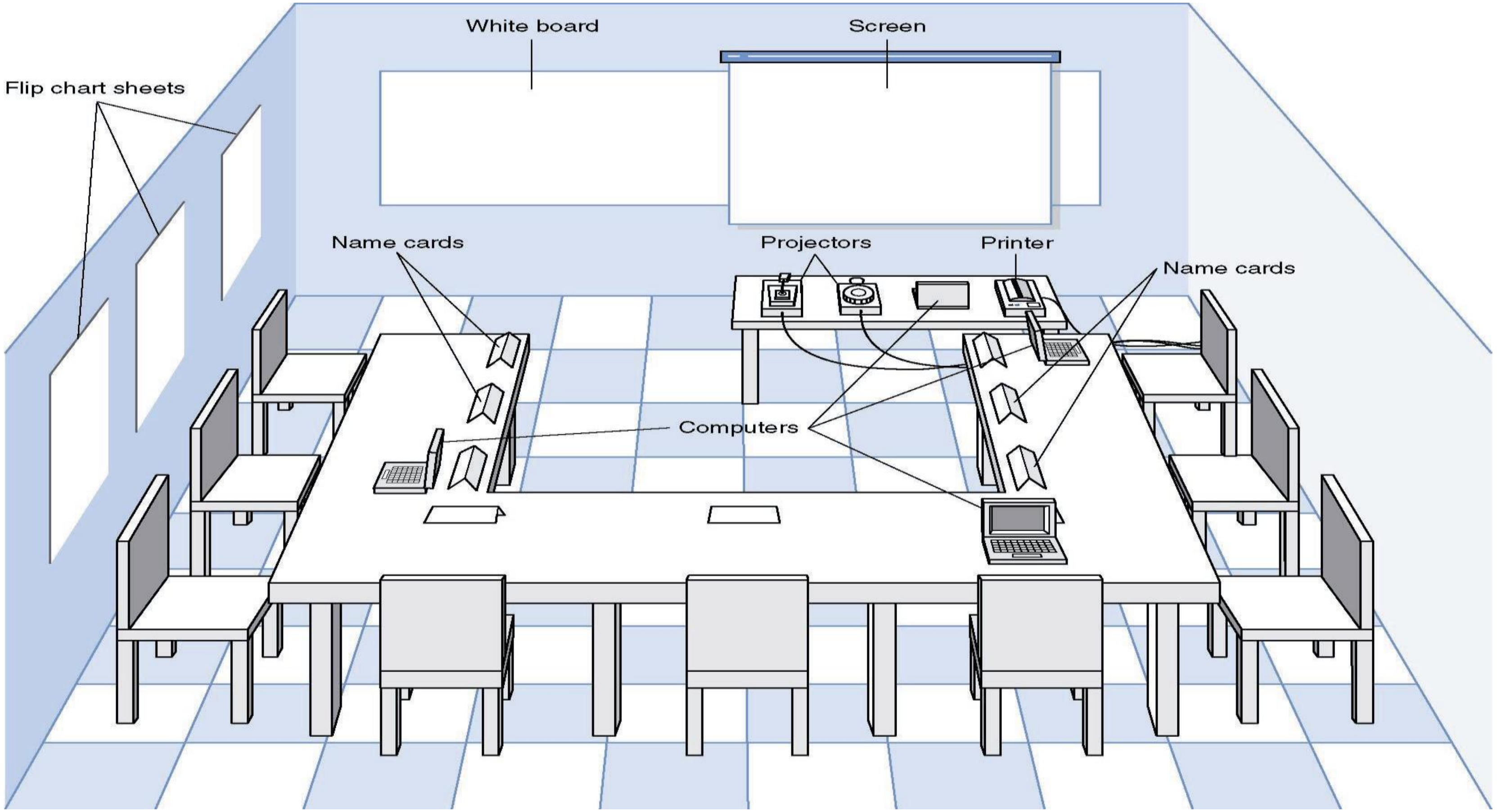
Joint Application Development (JAD)

- It is what the name implies — allows project managers, users, and developers to work together
- May reduce scope creep by 50%
- Avoids requirements being too specific or too vague

Joint application development (JAD)

- The membership
  - Leader (communication skills, not a stakeholder, knowledge of business domain)
  - Scribe (touch typing, software development knowledge, CASE tool skills)
  - Customers
    - Users
    - Managers
  - Developers

Joint Application Design (JAD) Setting



- U-Shaped seating
- Away from distractions
- Whiteboard/flip chart
- Prototyping tools

The JAD Session

- Tend to last 5 to 10 days over a three week period
- Prepare questions as with interviews
- Formal agenda
- Leader (Facilitator) activities
  - Keep session on track
  - Help with technical terms
  - Record group input
  - Help resolve issues
- Post-session follow-up

Software Requirements Specification

(SRS)

Software Requirements Specification (SRS)

- A tangible outcome of the requirements elicitation phase
- Produced according to an organization-approved template
- The software requirements specification is sometimes called
  - a functional specification,
  - a product specification,
  - a requirements document, or
  - a system specification

SRS – as in IEEE Standard 830-1998

- We will use templates derived from the one described in IEEE Standard 830-1998, "IEEE Recommended Practice for Software Requirements Specifications" (IEEE 1998b), *available in the course website*.

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Definitions, Acronyms and Abbreviations
    - 1.4 References
    - 1.5 Overview
2. Overall Description
    - 2.1 Use-Case Model Survey
        - 2.1.1 Introduction
        - 2.1.2 Survey Description
        - 2.1.3 Use-Case Model Hierarchy
        - 2.1.4 Diagrams of the Use-Case Model
    - 2.2 Assumptions and Dependencies
3. Specific requirements
    - (See next slide)

Appendixes
Index

Template of SRS Section 3 organized by user class

Table of Contents

1. Introduction
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Definitions, Acronyms and Abbreviations
   - 1.4 References
   - 1.5 Overview
2. Overall Description
   - 2.1 Use-Case Model Survey
     - 2.1.1 Introduction
     - 2.1.2 Survey Description
     - 2.1.3 Use-Case Model Hierarchy
     - 2.1.4 Diagrams of the Use-Case Model
   - 2.2 Assumptions and Dependencies

3. Specific requirements

Appendixes
Index

3. Requirements
   - 3.1 Use-Case Specifications
   - 3.2 Functionality
     - 3.2.1 <Functional Requirement One>
   - 3.3 Usability
     - 3.3.1 <Usability Requirement One>
   - 3.4 Reliability
   - 3.5 Performance
     - 3.5.1 <Performance Requirement One>
   - 3.6 Supportability
     - 3.6.1 <Supportability Requirement One>
   - 3.7 Design Constraints
     - 3.7.1 <Design Constraint One>
   - 3.8 Online User Documentation & Help System Requirements
   - 3.9 Purchased Components
   - 3.10 Interfaces
     - 3.10.1 User Interfaces
     - 3.10.2 Hardware Interfaces
     - 3.10.3 Software Interfaces
     - 3.10.4 Communications Interfaces
   - 3.11 Licensing Requirements
   - 3.12 Legal, Copyright and Other Notices
   - 3.13 Applicable Standards

3. Requirements

3.1 Use-Case Specifications

3.2 Functionality

3.2.1 <Functional Requirement One>

3.3 Usability

3.3.1 <Usability Requirement One>

3.4 Reliability

3.5 Performance

3.5.1 <Performance Requirement One>

3.6 Supportability

3.6.1 <Supportability Requirement One>

3.7 Design Constraints

3.7.1 <Design Constraint One>

3.8 Online User Documentation and Help System Requirements

3.9 Purchased Components

3.10 Interfaces

3.10.1 User Interfaces

3.10.2 Hardware Interfaces

3.10.3 Software Interfaces

3.10.4 Communications Interfaces

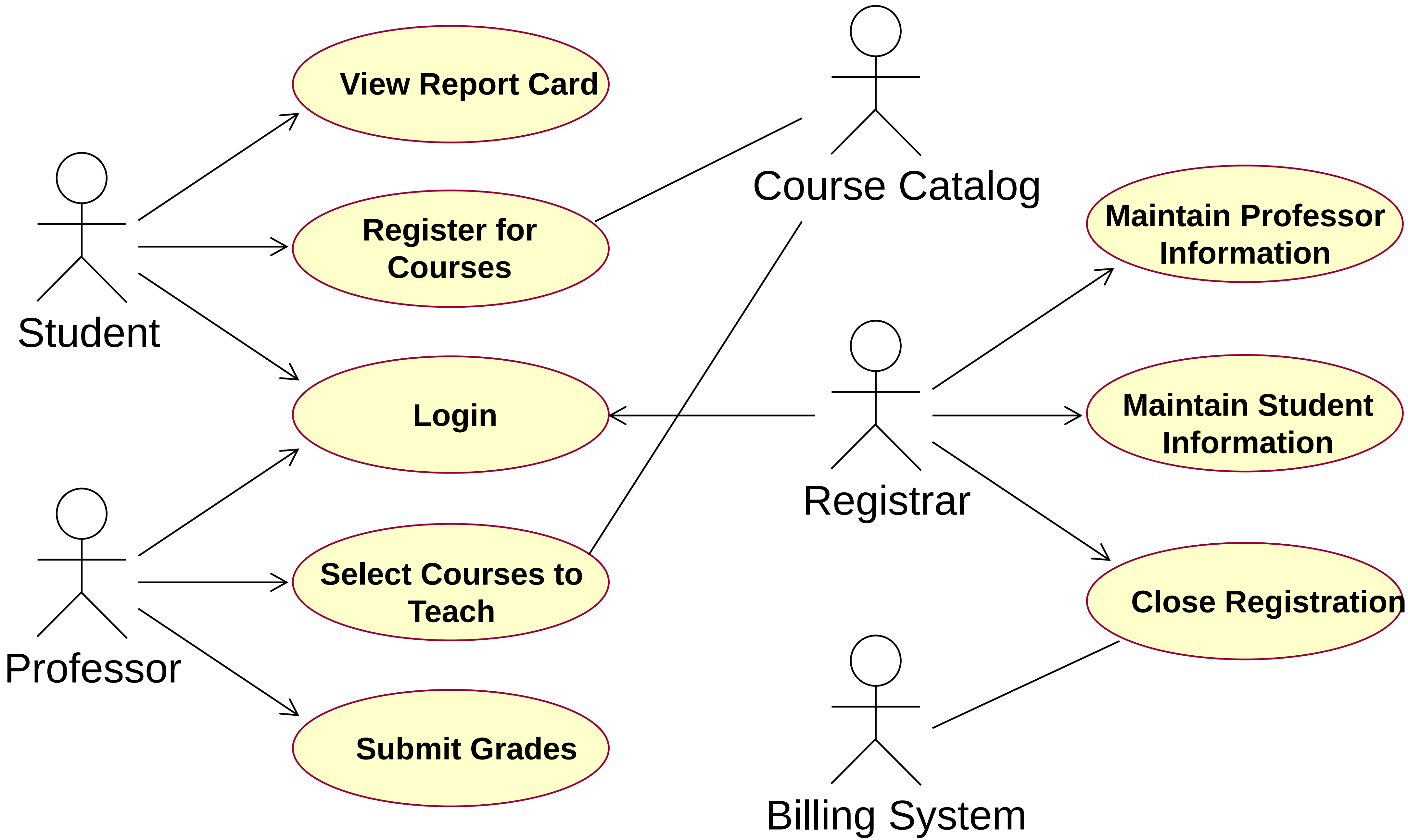
3.11 Licensing Requirements

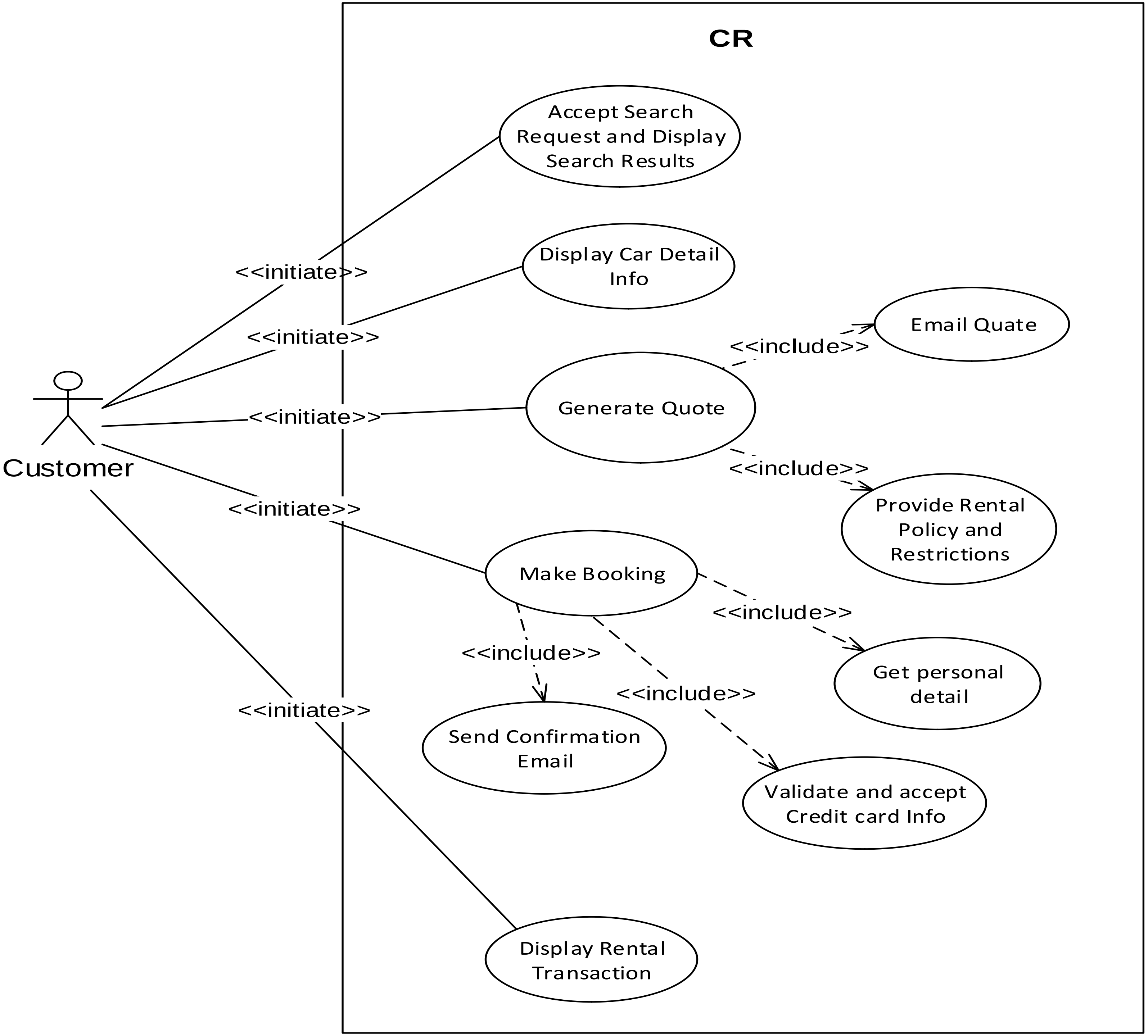
3.12 Legal, Copyright and Other Notices

3.13 Applicable Standards

Let us read UML

How Would You Read This Diagram?

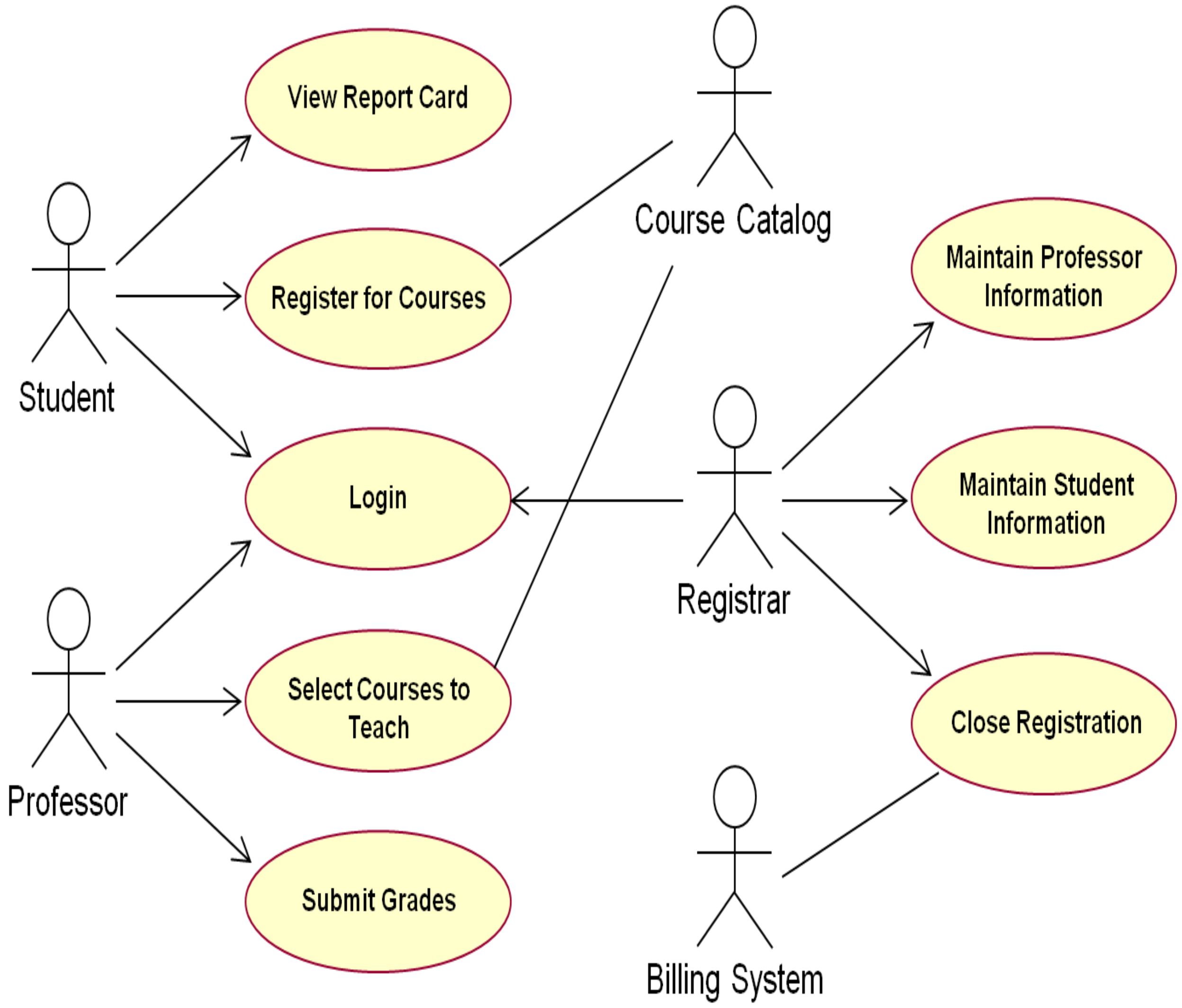




How Would You Read This Diagram?

How Would You Read This Diagram?

- Which use cases can a student perform? A professor? The Course Catalog?
- If Nasser is a student and professor, which use cases can he execute?
- Describe the functionality of this system.
- Describe the actor relationships for the Close Registration and Select Courses To Teach use cases.
- Which use case needs to run first, Register for Courses or View Report Card?



Questions?